
Documentação de Projeto

para o sistema

DevLevel

Versão 1.0

Projeto de sistema elaborado pelo aluno Karen Joilly Araújo Gregório de Almeida
como parte da disciplina **Projeto de Software**.

06 de Maio de 2026

1.	Introdução	2
2.	Modelos de Usuário e Requisitos	3
2.1	Descrição de Atores	3
2.2	Modelo de Casos de Uso.....	4
	Relacionamentos entre os Casos de Uso	6
	Diagrama de Casos de Uso.....	6
	Legenda do Diagrama.....	7
2.3	Diagrama de Sequência do Sistema	8
3.	Modelos de Projeto	12
3.1	Arquitetura	12
3.2	Diagrama de Componentes e Implantação.	12
3.2.1	Visão Geral da Arquitetura.....	13
3.2.2	Diagrama de Componentes e Implantação	13
3.3	Diagrama de Classes	15
3.3.1	Visão Geral do Modelo de Classes.....	15
3.3.2	Diagrama de Classes.....	15
3.3.3	Resumo das Classes por Categoria	16
3.4	Diagramas de Sequência	16
3.5	Diagramas de Comunicação	29
3.6	Diagramas de Estados	32
4.	Modelos de Dados.....	37

Tabela de Conteúdo

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Karen Joilly	07/05/26	Criação do Documento	0.1
Karen Joilly	14/05/26	Inclusão do Diagrama de Casos de Uso e Diagrama de Classes	0.2
Karen Joilly	15/05/26	Inclusão dos Diagramas de Sequência	0.3
Karen Joilly	19/05/26	Inclusão dos Diagramas de Estados	0.4
Karen Joilly	20/05/26	Inclusão dos Diagramas de Comunicação e Modelos de Dados	0.5
Karen Joilly	21/05/26	Revisão geral e finalização do documento para entrega	1.0

1. Introdução

Este documento agrega: 1) a elaboração e revisão de modelos de domínio e 2) modelos de projeto para o sistema DevLevel - Plataforma de Gamificação para Academia de Programação. A referência principal para a descrição geral do problema, domínio e requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema, disponível separadamente. O DevLevel foi concebido para resolver o problema de evasão (45%) em academias de programação, utilizando mecânicas de gamificação como XP (Experience Points), sistema de níveis, missões diárias personalizadas, trilhas de especialização, desbloqueio progressivo de conteúdo, ranqueamento semanal, loja de vantagens com moeda virtual (DevCoins) e sistema de streaks (ofensivas). O presente documento foca na documentação arquitetural e de design, detalhando os diagramas UML (Casos de Uso, Sequência, Classes, Estados, Componentes), os contratos de operação e as estratégias de mapeamento objeto-relacional.

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

Nesta subseção são apresentados e descritos os atores que interagem com o sistema DevLevel. Atores representam papéis desempenhados por usuários humanos ou sistemas externos que se comunicam com a plataforma.

Usuário: É o ator genérico que representa qualquer pessoa com cadastro ativo no sistema DevLevel. Este ator possui um perfil com informações básicas como nome, e-mail e data de registro. Todos os demais atores específicos herdam deste ator, compartilhando assim a capacidade de gerenciar seu próprio perfil e de se autenticar no sistema.

Aluno: Ator especializado que herda do ator Usuário. Representa a pessoa matriculada na academia de programação que utiliza o DevLevel para estudar, evoluir e competir. Suas principais interações com o sistema incluem evoluir de nível, realizar exercícios, assistir aulas, ajudar colegas no fórum e solicitar mentorias. O aluno é o ator central do sistema, sendo responsável pela maioria das atividades relacionadas à gamificação.

Mentor: Ator especializado que herda do ator Usuário. Representa o instrutor ou professor responsável por acompanhar turmas, gerenciar conquistas, gerenciar monitorias e exportar relatórios de desempenho. O mentor atua como facilitador do processo de aprendizagem, acompanhando o progresso dos alunos sob sua responsabilidade e intervindo quando necessário para validar conquistas especiais.

Administrador: Ator especializado que herda do ator Usuário. Representa o gestor do sistema com privilégios máximos, responsável pela configuração e manutenção da plataforma. Suas principais atribuições incluem gerenciar conteúdos didáticos, gerenciar o fórum, gerenciar badges (conquistas) e supervisionar o funcionamento geral do sistema.

2.2 Modelo de Casos de Uso

Nesta subseção é apresentado o diagrama de casos de uso do sistema DevLevel. Para cada caso de uso, foi atribuído um identificador único (ID) no formato UC-XX, que servirá como referência ao longo de todo o documento. O sistema é representado pelo retângulo que contém todos os casos de uso, sendo ele próprio o responsável pela execução das regras de negócio descritas.

Foram estabelecidas relações de herança entre os atores, de modo que os papéis específicos (Aluno, Mentor e Administrador) herdam as características do ator genérico Usuário, que representa qualquer pessoa com perfil cadastrado no sistema. Adicionalmente, foram utilizados relacionamentos de inclusão (<<include>>) para representar comportamentos obrigatórios e comuns a múltiplos casos de uso, como a autenticação, a validação de desbloqueio de conteúdos e o envio de notificações.

UC-01 – Evoluir de Nível: Este caso de uso tem como ator principal o Aluno. Seu objetivo é permitir que o aluno progrida em sua jornada de aprendizado ao acumular pontos de experiência (XP). O sistema monitora continuamente o XP total do aluno e, ao atingir o limite necessário, incrementa automaticamente seu nível em uma unidade. Este caso de uso inclui a autenticação do usuário e pode disparar notificações de evolução.

UC-02 – Gerenciar Turmas: Este caso de uso tem como ator principal o Mentor. Seu propósito é permitir que o instrutor visualize e administre as turmas sob sua responsabilidade, incluindo a gestão de alunos matriculados, acompanhamento de desempenho e organização de atividades. A autenticação é obrigatória para sua execução.

UC-03 – Gerenciar Conquistas: Este caso de uso tem como ator principal o Mentor. Seu objetivo é permitir que o instrutor valide e gerencie as conquistas obtidas pelos alunos, incluindo a aprovação manual de conquistas especiais que não puderam ser automaticamente detectadas pelo sistema. A autenticação é obrigatória para sua execução.

UC-04 – Gerenciar Conteúdos: Este caso de uso tem como ator principal o Administrador. Seu propósito é permitir que o gestor do sistema mantenha o catálogo de conteúdos didáticos disponíveis para os alunos, incluindo criar, editar, definir requisitos de nível e remover conteúdos. A autenticação é obrigatória para sua execução.

UC-05 – Gerenciar Perfil: Este caso de uso é comum a todos os atores que herdam do ator Usuário (Aluno, Mentor e Administrador). Seu objetivo é permitir que qualquer usuário autenticado visualize e atualize suas informações cadastrais, como nome, e-mail e senha. A autenticação é obrigatória para sua execução.

UC-06 – Autenticar: Este caso de uso é incluído (<<include>>) em todos os demais casos de uso que exigem que o usuário esteja devidamente identificado no sistema antes de executar qualquer operação. Seu objetivo é verificar as credenciais fornecidas pelo usuário (combinação de e-mail e senha) e conceder acesso ao sistema.

UC-07 – Ajudar Colega no Fórum: Este caso de uso tem como ator principal o Aluno. Seu objetivo é permitir que o aluno contribua com a comunidade respondendo dúvidas de outros alunos no fórum, ganhando XP por contribuições úteis. A autenticação é obrigatória para sua execução.

UC-08 – Assistir Aula: Este caso de uso tem como ator principal o Aluno. Seu propósito é permitir que o aluno assista a videoaulas e materiais didáticos disponíveis na plataforma, ganhando XP por tempo de dedicação ao estudo. Este caso de uso inclui a autenticação do usuário e a validação de desbloqueio do conteúdo conforme o nível do aluno.

UC-09 – Realizar Exercício: Este caso de uso tem como ator principal o Aluno. Seu objetivo é permitir que o aluno resolva exercícios de programação para praticar os conceitos aprendidos, recebendo XP conforme a dificuldade do exercício realizado. A autenticação e a validação de desbloqueio são obrigatórias para sua execução.

UC-10 – Exportar Relatório: Este caso de uso tem como ator principal o Mentor. Seu propósito é permitir que o instrutor gere e exporte relatórios de desempenho das turmas, com estatísticas como média de XP, distribuição de níveis e taxas de conclusão de atividades. A autenticação é obrigatória para sua execução.

UC-11 – Gerenciar Fórum: Este caso de uso tem como ator principal o Administrador. Seu objetivo é permitir que o gestor do sistema modere o fórum, incluindo a exclusão de conteúdo impróprio, edição de mensagens, bloqueio de usuários infratores e configuração das regras gerais de participação. A autenticação é obrigatória para sua execução.

UC-12 – Solicitar Mentoria: Este caso de uso tem como ator principal o Aluno. Seu propósito é permitir que o aluno solicite uma sessão de mentoria particular com um mentor, utilizando DevCoins como forma de pagamento. A autenticação é obrigatória para sua execução, e o sistema pode disparar notificações para manter o aluno informado sobre o status de sua solicitação.

UC-13 – Notificar: Este caso de uso é incluído (<<include>>) em outros casos de uso que necessitam enviar comunicações aos usuários. Seu objetivo é encapsular a lógica de envio de notificações (e-mail, push, SMS) sobre eventos relevantes como evolução de nível e respostas a solicitações de mentoria.

UC-14 – Gerenciar Badges: Este caso de uso tem como ator principal o Administrador. Seu propósito é permitir que o gestor do sistema crie, edite, ative, desative ou remova badges (conquistas) disponíveis para os alunos na plataforma. A autenticação é obrigatória para sua execução.

UC-15 – Gerenciar Monitoria: Este caso de uso tem como ator principal o Mentor. Seu objetivo é permitir que o instrutor gerencie as solicitações de mentoria recebidas dos alunos, incluindo aceitar, recusar, remarcar ou cancelar os agendamentos. A autenticação é obrigatória para sua execução.

UC-16 – Validar Desbloqueio: Este caso de uso é incluído (<<include>>) nos casos de uso que envolvem acesso a conteúdos restritos por nível, como assistir aula (UC-08) e realizar exercício (UC-09). Seu objetivo é verificar se o aluno possui nível suficiente para acessar determinado conteúdo, garantindo a progressão adequada na trilha de aprendizado.

Relacionamentos entre os Casos de Uso

O diagrama de casos de uso estabelece os seguintes relacionamentos:

Relações de Herança entre Atores: Os atores Aluno, Mentor e Administrador herdam do ator genérico Usuário. Esta modelagem indica que todos compartilham a capacidade de gerenciar seu próprio perfil (UC-05) e de se autenticar no sistema (UC-06).

Relações de Inclusão (<<include>>): Todos os casos de uso que exigem que o usuário esteja identificado incluem obrigatoriamente o caso de uso UC-06 (Autenticar). Adicionalmente, os casos de uso UC-08 (Assistir Aula) e UC-09 (Realizar Exercício) incluem o UC-16 (Validar Desbloqueio) para verificar se o aluno possui nível suficiente. Os casos de uso UC-01 (Evoluir de Nível), UC-12 (Solicitar Mentoria) e outros que geram comunicações incluem o UC-13 (Notificar) para o envio de alertas aos usuários.

Diagrama de Casos de Uso

O diagrama de casos de uso do sistema DevLevel é apresentado na Figura 1, onde é possível visualizar a hierarquia de atores, os relacionamentos de herança, e as conexões de inclusão e extensão entre os casos de uso.



Figura 1 - Diagrama de Casos de Uso do Sistema DevLevel

Legenda do Diagrama

- **Herança entre atores:** Aluno, Mentor e Administrador são especializações do ator genérico Usuário. Esta modelagem indica que todos compartilham a capacidade de gerenciar seu próprio perfil (UC-05).
- **Include (<<include>>):** Todos os casos de uso que exigem que o usuário esteja identificado incluem obrigatoriamente o caso de uso UC-06 (Autenticar). Isto significa que a autenticação é uma pré-condição obrigatória para a execução dessas funcionalidades.
- **Extend (<<extend>>):** As extensões representam comportamentos opcionais. O sistema pode, opcionalmente, notificar o aluno (UC-13) quando ele evolui de nível (UC-01) ou quando sua solicitação de mentoria (UC-12) é respondida.

2.3 Diagrama de Sequência do Sistema

Contrato	CT-01
Operação	evoluirNivel(alunoId: Long, xpGanho: Integer): ResultadoEvolucao
Referências cruzadas	UC-01 - Evoluir de Nível
Pré-condições	1. O aluno deve estar autenticado no sistema. 2. O aluno deve ter realizado alguma atividade que gere XP. 3. O XP total do aluno deve ser conhecido pelo sistema.
Pós-condições	1. Se XP total $\geq$ XP necessário para próximo nível, o nível do aluno é incrementado. 2. O XP remanescente é calculado e armazenado. 3. Conteúdos com requisito de nível igual ao novo nível são desbloqueados. 4. Badges relacionadas a marcos de nível são concedidas. 5. Uma notificação de evolução é enviada ao aluno. 6. O histórico de evolução é registrado.

Contrato	CT-02
Operação	gerenciarTurma(mentorId: Long, turmaId: Long, acao: String, dados: Map): ResultadoOperacao
Referências cruzadas	UC-02 - Gerenciar Turmas
Pré-condições	1. O mentor deve estar autenticado no sistema. 2. O mentor deve ser responsável pela turma informada. 3. A turma deve existir e estar ativa. 4. Para adicionar aluno: o email do aluno deve ser válido. 5. Para remover aluno: o aluno deve estar matriculado na turma.
Pós-condições	1. Ao listar turmas: o sistema retorna todas as turmas sob responsabilidade do mentor. 2. Ao selecionar turma: o sistema exibe dados completos da turma. 3. Ao adicionar aluno: uma nova matrícula é criada no sistema. 4. Ao remover aluno: a matrícula é desativada ou removida. 5. Todas as operações são registradas em log.

Contrato	CT-03
Operação	validarConquista(mentorId: Long, solicitacaoId: Long, acao: String, observacao: String): ResultadoValidacao
Referências cruzadas	UC-03 - Gerenciar Conquistas
Pré-condições	1. O mentor deve estar autenticado no sistema. 2. O mentor deve ter permissão para validar conquistas. 3. A solicitação deve existir e estar com status "PENDENTE". 4. Para aprovação: as evidências devem ser válidas. 5. Para reprovação: um motivo deve ser fornecido.
Pós-condições	1. Se aprovada: o badge é concedido ao aluno. 2. Se aprovada: o XP correspondente é adicionado ao aluno. 3. Se aprovada: uma notificação de conquista é enviada ao aluno. 4. Se reprovada: o aluno é notificado com o motivo. 5. O status da solicitação é atualizado para "APROVADA" ou "REPROVADA". 6. A ação é registrada no histórico do mentor.

Contrato	CT-04
Operação	gerenciarConteudo(adminId: Long, acao: String, dadosConteudo: Map, arquivo: File): ResultadoConteudo
Referências cruzadas	UC-04 - Gerenciar Conteúdos
Pré-condições	1. O administrador deve estar autenticado no sistema. 2. O administrador deve ter permissão de "ADMIN_CONTEUDO". 3. Para criação: todos os campos obrigatórios devem ser preenchidos. 4. Para edição: o conteúdo deve existir. 5. Para remoção: o conteúdo não deve estar em uso por alunos.
Pós-condições	1. Ao criar: um novo conteúdo é adicionado ao catálogo. 2. Ao editar: os dados do conteúdo são atualizados. 3. Ao remover: o conteúdo é removido ou arquivado. 4. Um log da operação é registrado. 5. O administrador recebe confirmação da operação.

Contrato	CT-05
Operação	gerenciarPerfil(usuarioId: Long, acao: String, dados: Map): ResultadoPerfil
Referências cruzadas	UC-05 - Gerenciar Perfil
Pré-condições	1. O usuário deve estar autenticado no sistema. 2. Para edição de perfil: os novos dados devem ser válidos. 3. Para alteração de senha: a senha atual deve ser fornecida corretamente. 4. Para alteração de senha: a nova senha deve atender aos critérios de segurança (mínimo 8 caracteres, letras e números).
Pós-condições	1. Ao visualizar: o sistema exibe os dados atuais do usuário. 2. Ao editar: os dados do perfil são atualizados no sistema. 3. Ao alterar senha: a nova senha é criptografada e armazenada. 4. O usuário recebe confirmação das alterações. 5. As alterações são registradas em log de auditoria

Contrato	CT-06
-----------------	-------

Documentação de Projeto para o Sistema DevLevel Página

Operação	ajudarForum(alunoId: Long, perguntaId: Long, respostaId: Long, acao: String, conteudo: String): ResultadoForum
Referências cruzadas	UC-07 - Ajudar Colega no Fórum
Pré-condições	1. O aluno deve estar autenticado no sistema. 2. Para responder: a pergunta deve estar aberta e sem resposta aceita. 3. Para responder: o conteúdo deve ter no mínimo 20 caracteres. 4. Para marcar melhor resposta: o aluno deve ser o autor da pergunta. 5. Para marcar melhor resposta: a resposta deve existir.
Pós-condições	1. Ao responder: uma nova resposta é registrada no fórum. 2. Ao responder: o aluno ganha 15 XP por ajudar. 3. Ao responder: o autor da pergunta é notificado. 4. Ao marcar melhor resposta: o respondente ganha 25 XP bônus. 5. Ao marcar melhor resposta: a pergunta é marcada como "CONCLUÍDA". 6. O histórico de participação no fórum é atualizado.

Contrato	CT-07
Operação	assistirAula(alunoId: Long, aulaId: Long, acao: String, tempoDecorrido: Integer): ResultadoAula
Referências cruzadas	UC-08 - Assistir Aula
Pré-condições	1. O aluno deve estar autenticado no sistema. 2. O aluno deve ter nível mínimo para acessar a aula. 3. A aula deve existir no catálogo de conteúdos. 4. Para conclusão: o aluno deve ter assistido pelo menos 90% da aula.
Pós-condições	1. Ao acessar: o sistema registra o início da visualização. 2. Durante: o progresso é atualizado periodicamente. 3. Ao concluir com sucesso: o aluno ganha XP proporcional à duração e nível da aula. 4. Ao concluir com sucesso: a aula é marcada como concluída no histórico. 5. O aluno recebe feedback sobre a conclusão. 6. Se o XP acumulado atingir novo nível, o UC-01 é disparado.

Contrato	CT-08
Operação	realizarExercicio(alunoId: Long, exercicioId: Long, codigo: String): ResultadoExercicio
Referências cruzadas	UC-09 - Realizar Exercício
Pré-condições	1. O aluno deve estar autenticado no sistema. 2. O aluno deve ter nível mínimo para acessar o exercício. 3. O exercício deve existir no catálogo. 4. O código submetido deve estar em formato válido.
Pós-condições	1. Se correto: o XP do aluno é incrementado (10 para básico, 25 para intermediário, 50 para avançado). 2. Se correto: um registro de acerto é adicionado ao histórico. 3. Se incorreto: um registro de erro é adicionado ao histórico, sem ganho de XP. 4. O aluno recebe feedback imediato sobre sua submissão. 5. O sistema verifica se o aluno atingiu XP para evoluir de nível (disparando UC-01). 6. Se o exercício faz parte de uma missão diária, a missão é atualizada.

Contrato	CT-09
-----------------	-------

Documentação de Projeto para o Sistema DevLevel Página

Operação	exportarRelatorio(mentorId: Long, tipoRelatorio: String, parametros: Map, formato: String): ResultadoExportacao
Referências cruzadas	UC-10 - Exportar Relatório
Pré-condições	1. O mentor deve estar autenticado no sistema. 2. O mentor deve ter acesso aos dados solicitados (turmas sob sua responsabilidade). 3. O tipo de relatório deve ser válido. 4. O formato deve ser suportado (PDF, CSV, EXCEL). 5. Os parâmetros de filtro devem ser válidos.
Pós-condições	1. Os dados são coletados do banco de dados conforme os filtros. 2. Um arquivo no formato solicitado é gerado. 3. Um link temporário para download é disponibilizado. 4. A exportação é registrada em log. 5. Se agendado: o relatório será enviado periodicamente por email. 6. O mentor recebe confirmação com o link para download.

Contrato	CT-10
Operação	gerenciarForum(adminId: Long, acao: String, conteudoId: Long, dados: Map): ResultadoModeracao
Referências cruzadas	UC-11 - Gerenciar Fórum
Pré-condições	1. O administrador deve estar autenticado no sistema. 2. O administrador deve ter permissão de moderação. 3. Para exclusão/edição: o conteúdo deve existir. 4. Para bloqueio: o usuário deve existir e não estar bloqueado. 5. Para configuração: a chave deve ser válida e o valor dentro dos limites.
Pós-condições	1. Ao excluir: o conteúdo é removido do fórum e o autor é notificado. 2. Ao editar: o conteúdo é atualizado com registro de edição. 3. Ao bloquear: o usuário perde acesso ao fórum e seu conteúdo é removido. 4. Ao reabilitar: o acesso do usuário é restaurado. 5. Ao configurar: as regras do fórum são atualizadas. 6. Todas as ações são registradas em log de moderação.

Contrato	CT-11
Operação	solicitarMentoria(alunoId: Long, mentorId: Long, dataHora: DateTime, mensagem: String): ResultadoMentoria
Referências cruzadas	UC-12 - Solicitar Mentoria
Pré-condições	1. O aluno deve estar autenticado no sistema. 2. O mentor deve existir e estar ativo. 3. O horário deve estar dentro da disponibilidade do mentor. 4. O aluno deve ter saldo mínimo de 100 DevCoins. 5. O aluno não pode ter solicitação pendente para o mesmo mentor no mesmo horário.
Pós-condições	1. Se mentor disponível e saldo suficiente: uma solicitação PENDENTE é criada. 2. 100 DevCoins são debitados do saldo do aluno. 3. O mentor é notificado sobre a nova solicitação. 4. O aluno recebe confirmação da solicitação. 5. Se mentor recusar posteriormente: os DevCoins são estornados. 6. Se mentor aceitar: link de videoconferência é gerado e compartilhado.

Contrato	CT-12
Operação	gerenciarBadges(adminId: Long, acao: String, badgeId: Long, dadosBadge: Map, icone: File): ResultadoBadge
Referências cruzadas	UC-14 - Gerenciar Badges
Pré-condições	1. O administrador deve estar autenticado no sistema. 2. O administrador deve ter permissão de "ADMIN_BADGES". 3. Para criação: nome único, critério válido, XP entre 50 e 1000. 4. Para edição: o badge deve existir. 5. Para remoção: o badge não deve estar em uso ou será apenas arquivado.
Pós-condições	1. Ao criar: um novo badge é adicionado ao catálogo. 2. Ao editar: os dados do badge são atualizados. 3. Ao desativar: o badge não pode mais ser concedido, mas permanece para quem já o possui. 4. Ao reativar: o badge volta a estar disponível para novas concessões. 5. Ao remover: o badge é arquivado ou removido. 6. Um log da operação é registrado.

Contrato	CT-13
Operação	gerenciarMonitoria(mentorId: Long, solicitacaoId: Long, acao: String, dados: Map): ResultadoMonitoria
Referências cruzadas	UC-15 - Gerenciar Monitoria
Pré-condições	1. O mentor deve estar autenticado no sistema. 2. O mentor deve ser o responsável pela solicitação. 3. A solicitação deve estar com status "PENDENTE". 4. Para aceitação: o horário deve continuar disponível. 5. Para recusa: um motivo deve ser fornecido. 6. Para remarcação: a nova data deve ser válida.
Pós-condições	1. Ao aceitar: a solicitação passa para "ACEITA" e um link de videoconferência é gerado. 2. Ao aceitar: o aluno é notificado com o link. 3. Ao recusar: a solicitação passa para "RECUSADA". 4. Ao recusar: os 100 DevCoins são estornados para o aluno. 5. Ao recusar: o aluno é notificado com o motivo. 6. Ao remarcar: a data da mentoria é atualizada. 7. Todas as ações são registradas no histórico.

3. Modelos de Projeto

3.1 Arquitetura

Pode ser descrita com um diagrama apropriado da UML ou C4 Model

3.2 Diagrama de Componentes e Implantação.

Nesta subseção são apresentados os diagramas de componentes e implantação do sistema DevLevel, mostrando a arquitetura distribuída em nuvem AWS com microsserviços, filas, banco de dados PostgreSQL e serviços de notificação. O diagrama contempla tanto a visão estrutural dos componentes quanto sua alocação física na infraestrutura de nuvem.

3.2.1 Visão Geral da Arquitetura

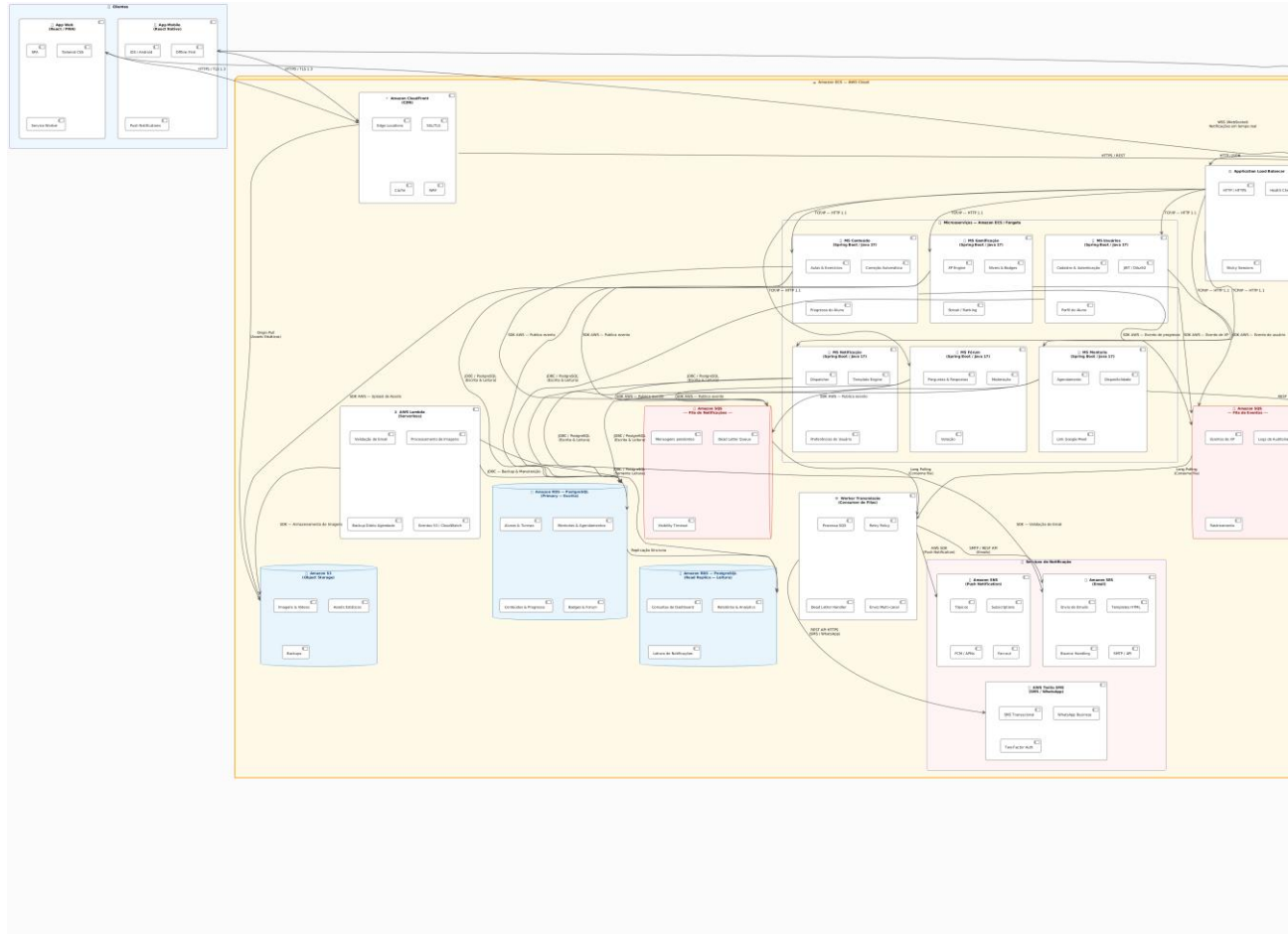
O sistema DevLevel adota uma arquitetura baseada em microsserviços implantados na Amazon ECS (Elastic Container Service) com orquestração via AWS Fargate. A comunicação externa é realizada através de Amazon CloudFront como CDN e Amazon API Gateway como ponto único de entrada, que roteia as requisições para o Application Load Balancer (ALB). Este, por sua vez, distribui o tráfego entre os seis microsserviços que compõem a lógica de negócio do sistema.

A comunicação assíncrona é realizada por meio de filas Amazon SQS, que desacoplam o processamento de notificações e eventos. Um Worker Transmissão consome essas filas e dispara os canais de notificação apropriados: Amazon SES para e-mails, Amazon SNS para push notifications e Twilio para SMS/WhatsApp. Funções AWS Lambda são utilizadas para tarefas serverless como validação de e-mail, processamento de imagens e backup diário.

O banco de dados principal é o Amazon RDS PostgreSQL em configuração Primary para operações de escrita e Read Replica para consultas de leitura, otimizando o desempenho de dashboards e relatórios. O Amazon S3 armazena objetos estáticos como imagens, vídeos e assets. O Amazon Location Service fornece funcionalidades de geolocalização para recursos de localização.

3.2.2 Diagrama de Componentes e Implantação

O diagrama a seguir apresenta a arquitetura completa do sistema, incluindo os componentes internos (microsserviços, filas, workers, lambdas, bancos de dados) e sua implantação na nuvem AWS ECS, bem como os clientes externos (Web e Mobile) que acessam o sistema via CloudFront e API Gateway.



Camada

Componentes

Apresentação

React Web, React Native, CloudFront

Gateway

API Gateway, ALB

Aplicação (Microserviços)

Usuários, Gamificação, Conteúdo, Mentoria, Fórum, Notificação

Processamento Assíncrono

SQS, Worker Transmissão, Lambda

Notificações

SES, SNS, Twilio

Dados

RDS Primary, RDS Read Replica, S3

3.3 Diagrama de Classes

Nesta subseção é apresentado o diagrama de classes do sistema DevLevel, que representa a estrutura estática do sistema, incluindo as classes, seus atributos, métodos e os relacionamentos entre elas. O diagrama segue os princípios da orientação a objetos e incorpora padrões de projeto apropriados para o domínio de gamificação.

3.3.1 Visão Geral do Modelo de Classes

O modelo de classes do DevLevel é organizado em torno do conceito central de Usuário, que é uma classe abstrata da qual derivam os papéis específicos de Aluno, Mentor e Administrador. A interface Autenticavel define o contrato para objetos que podem ser autenticados no sistema, sendo implementada pela classe Usuario.

As principais entidades de domínio incluem Turma (agrupamento de alunos), Matricula (classe associativa entre Aluno e Turma), Badge (conquistas desbloqueáveis), Conteudo (material didático), MissaoDiaria (desafios diários), Mentoria (sessões de mentoria) e Notificacao (comunicações enviadas aos usuários).

Foram utilizados enums para representar conjuntos de valores fixos e bem definidos, como TipoNotificacao (evolução, mentoria, conquista, alerta), StatusMentoria (pendente, aceita, recusada, concluída, cancelada) e TrilhaEspecializacao (front-end, back-end, dados, mobile, devops).

3.3.2 Diagrama de Classes

O diagrama a seguir apresenta todas as classes do sistema DevLevel, seus atributos, métodos e os relacionamentos de herança, associação, composição e dependência entre elas.

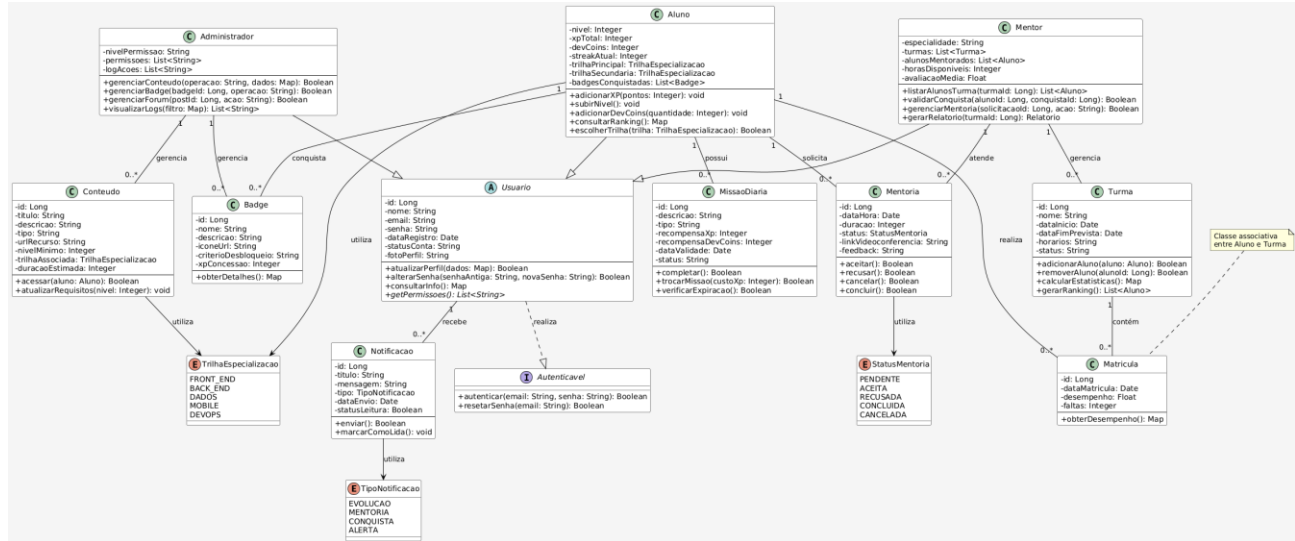


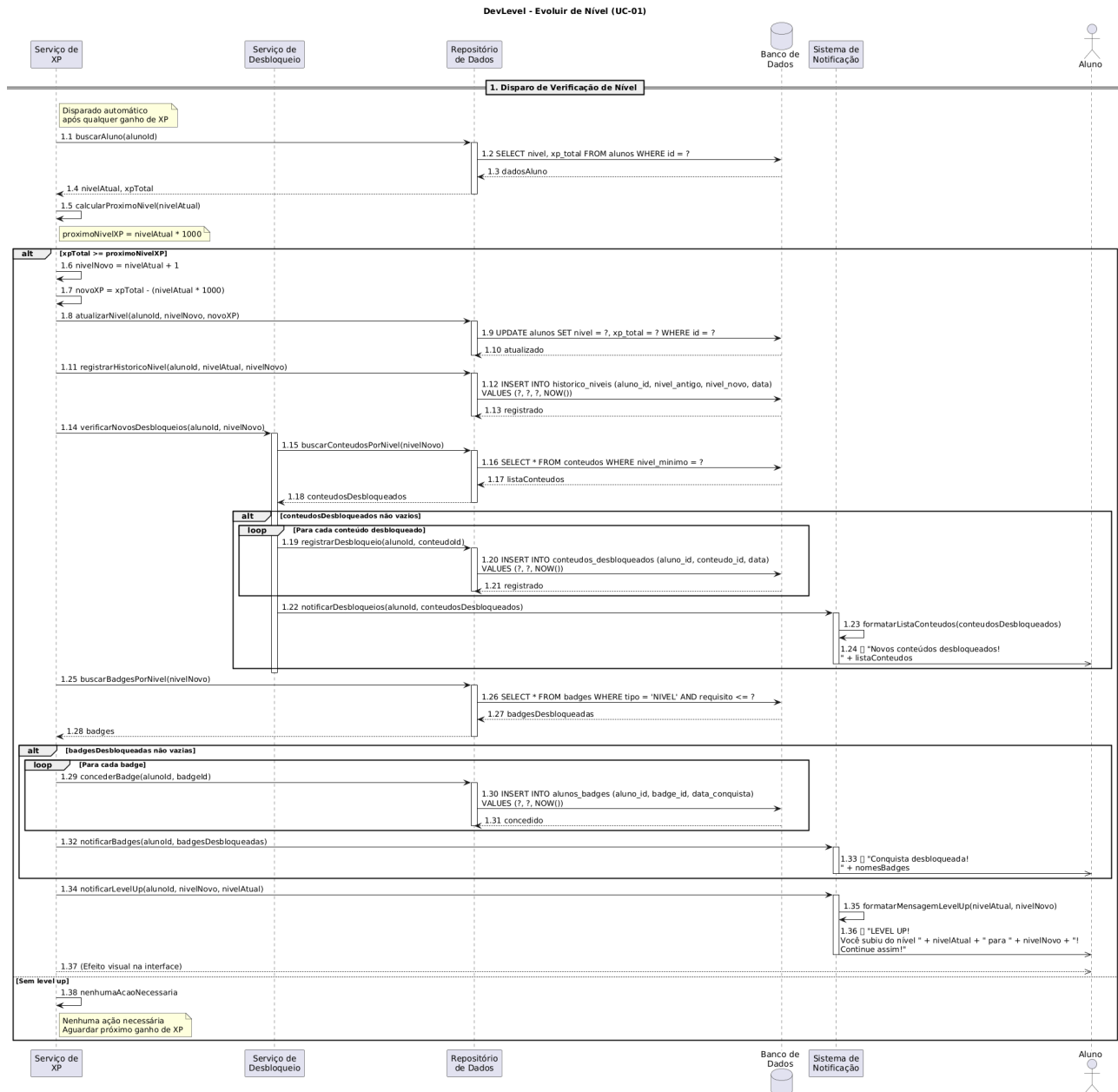
Figura 3.3.1: Diagrama de Classes do Sistema DevLevel

3.3.3 Resumo das Classes por Categoria

Categoria	Classes
Interface	Autenticavel
Abstrata	Usuario
Usuários	Aluno, Mentor, Administrador
Domínio	Turma, Badge, Conteudo, MissaoDiaria, Mentoria, Notificacao
Associativa	Matricula
Enums	TipoNotificacao, StatusMentoria, TrilhaEspecializacao

3.4 Diagramas de Sequência

O diagrama a seguir detalha o processo automático de evolução de nível do aluno, que é disparado sempre que há ganho de XP. O sistema verifica se o XP total atingiu o limite necessário para o próximo nível e, em caso positivo, atualiza o nível, desbloqueia conteúdos, concede badges e dispara notificações.



3.4.1 Diagrama de Sequência do Sistema - UC-01: Evoluir de Nível

O diagrama a seguir detalha as interações para o mentor gerenciar suas turmas, incluindo visualização da lista de turmas, exibição de detalhes, adição e remoção de alunos.

Documentação de Projeto para o Sistema DevLevel Página

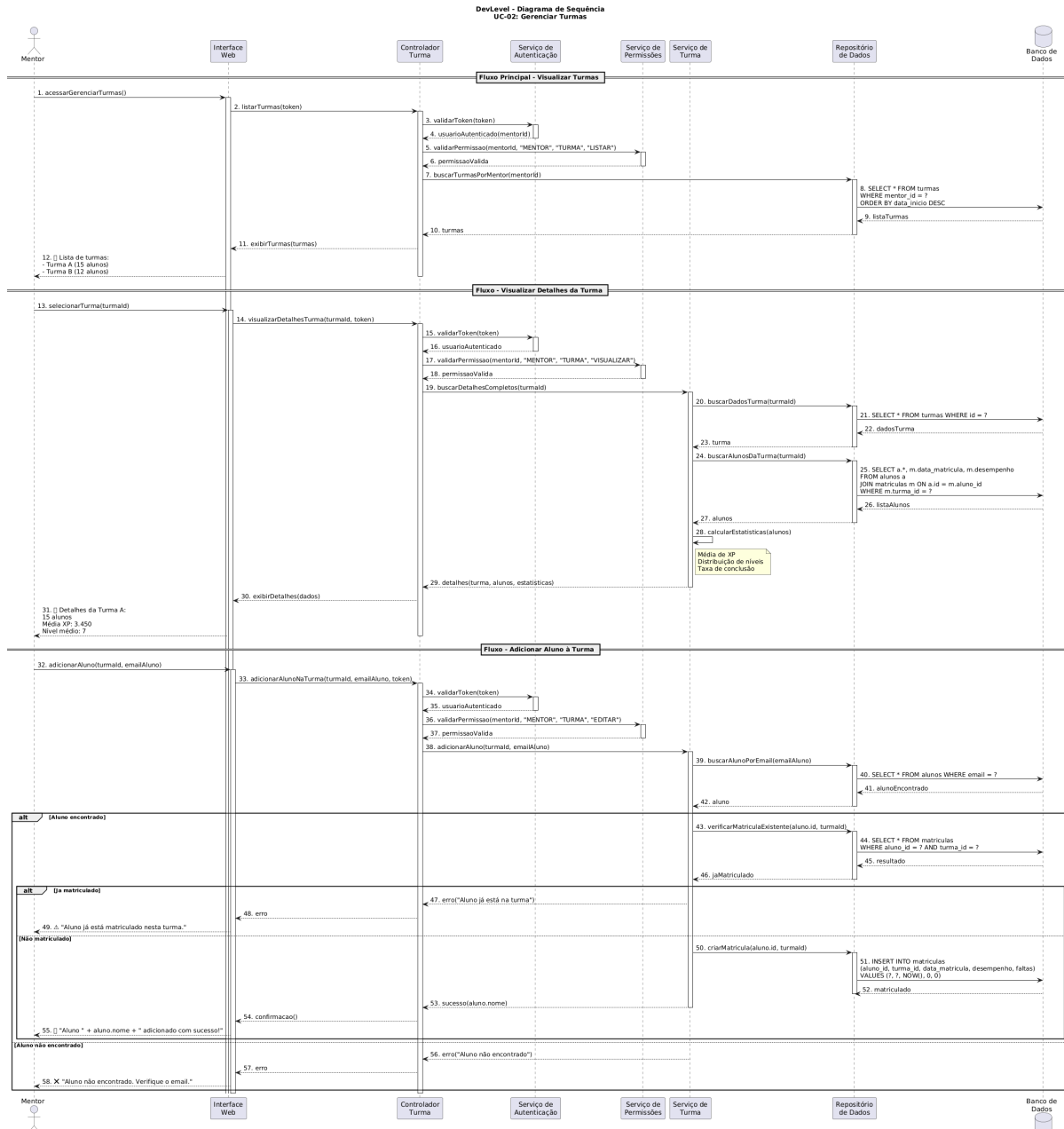


Figura - Diagrama de Sequência do Sistema - UC-02: Gerenciar Turmas

O diagrama a seguir detalha o processo de validação manual de conquistas pelo mentor, incluindo a visualização de solicitações pendentes, aprovação com concessão de badge e XP, reprovação com notificação do aluno, e registro de todas as ações.

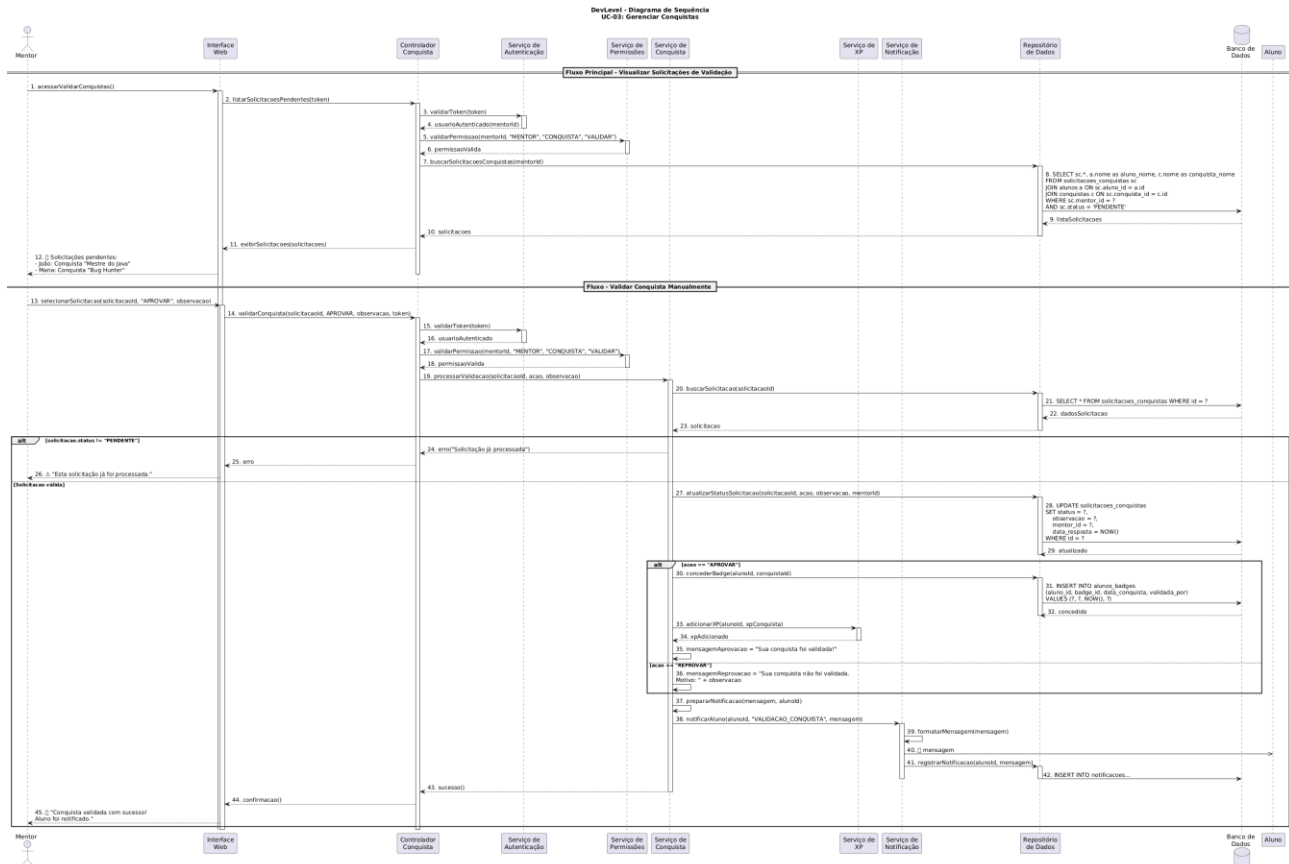


Figura - Diagrama de Sequência do Sistema - UC-03: Gerenciar Conquistas

O diagrama a seguir detalha as operações de gerenciamento de conteúdos didáticos pelo administrador, incluindo criação, edição, visualização e remoção de conteúdos, bem como a pré-visualização opcional antes da publicação.

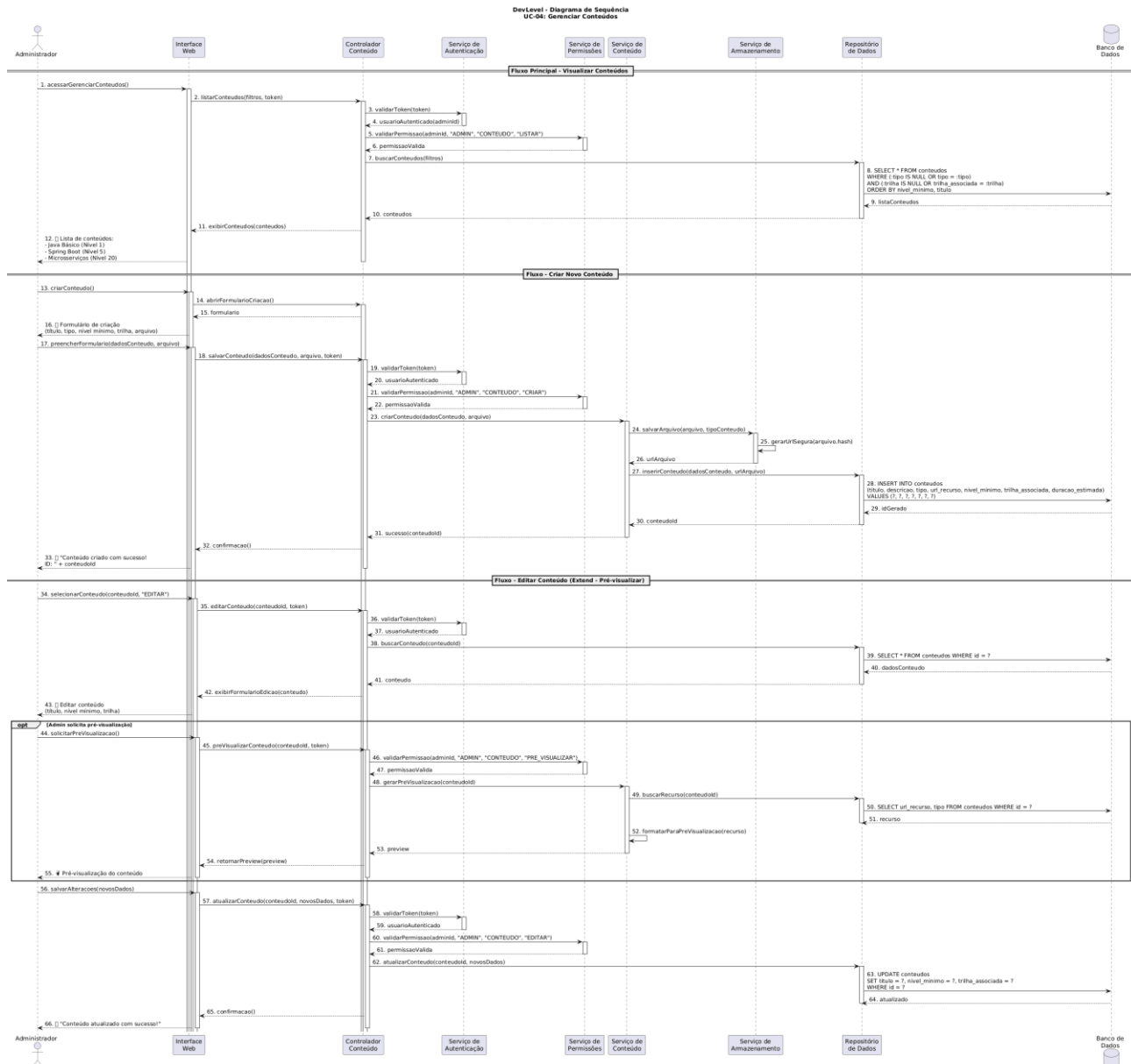


Figura - Diagrama de Sequência do Sistema - UC-04: Gerenciar Conteúdos

O diagrama a seguir detalha as operações de gerenciamento de perfil disponíveis para todos os tipos de usuário, incluindo visualização de dados, edição de informações cadastrais, alteração de senha e solicitação opcional de exclusão de conta.

Documentação de Projeto para o Sistema DevLevel Página

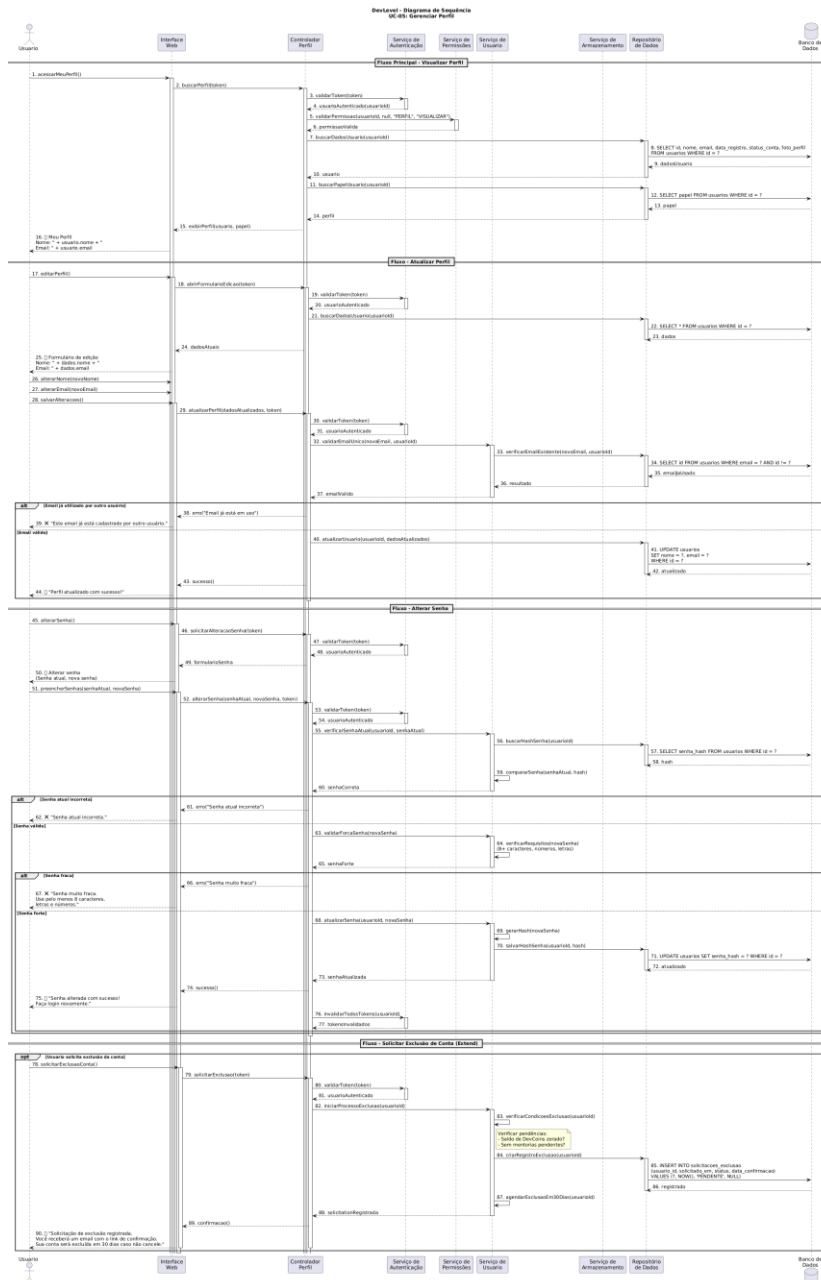


Figura - Diagrama de Sequência do Sistema - UC-05: Gerenciar Perfil

O diagrama a seguir detalha a interação do aluno com o fórum, incluindo visualização de perguntas pendentes, envio de respostas com validação de qualidade, ganho de XP por ajuda, e a funcionalidade de marcar a melhor resposta com bônus adicional.



O diagrama a seguir detalha o fluxo de acesso e reprodução de aulas pelo aluno, incluindo validação de desbloqueio do conteúdo, acompanhamento de progresso em tempo real, e concessão de XP ao concluir a aula com percentual mínimo assistido.

Documentação de Projeto para o Sistema DevLevel Página

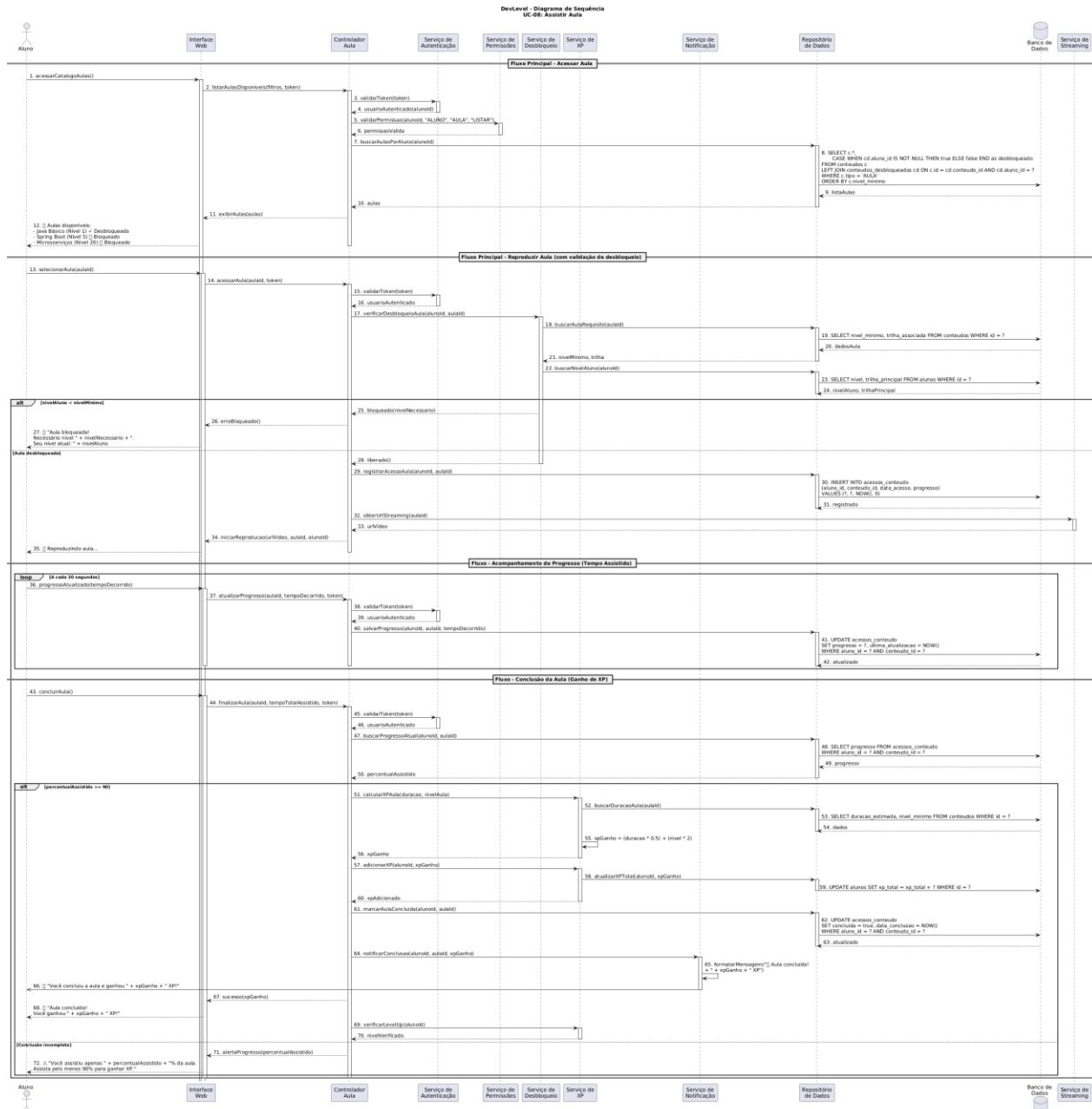


Figura - Diagrama de Sequência do Sistema - UC-08: Assistir Aula

O diagrama a seguir detalha o processo de submissão e correção de exercícios de programação pelo aluno, incluindo validação de desbloqueio, correção automática do código, cálculo e concessão de XP conforme a dificuldade, e registro do histórico de acertos e erros.

Documentação de Projeto para o Sistema DevLevel Página

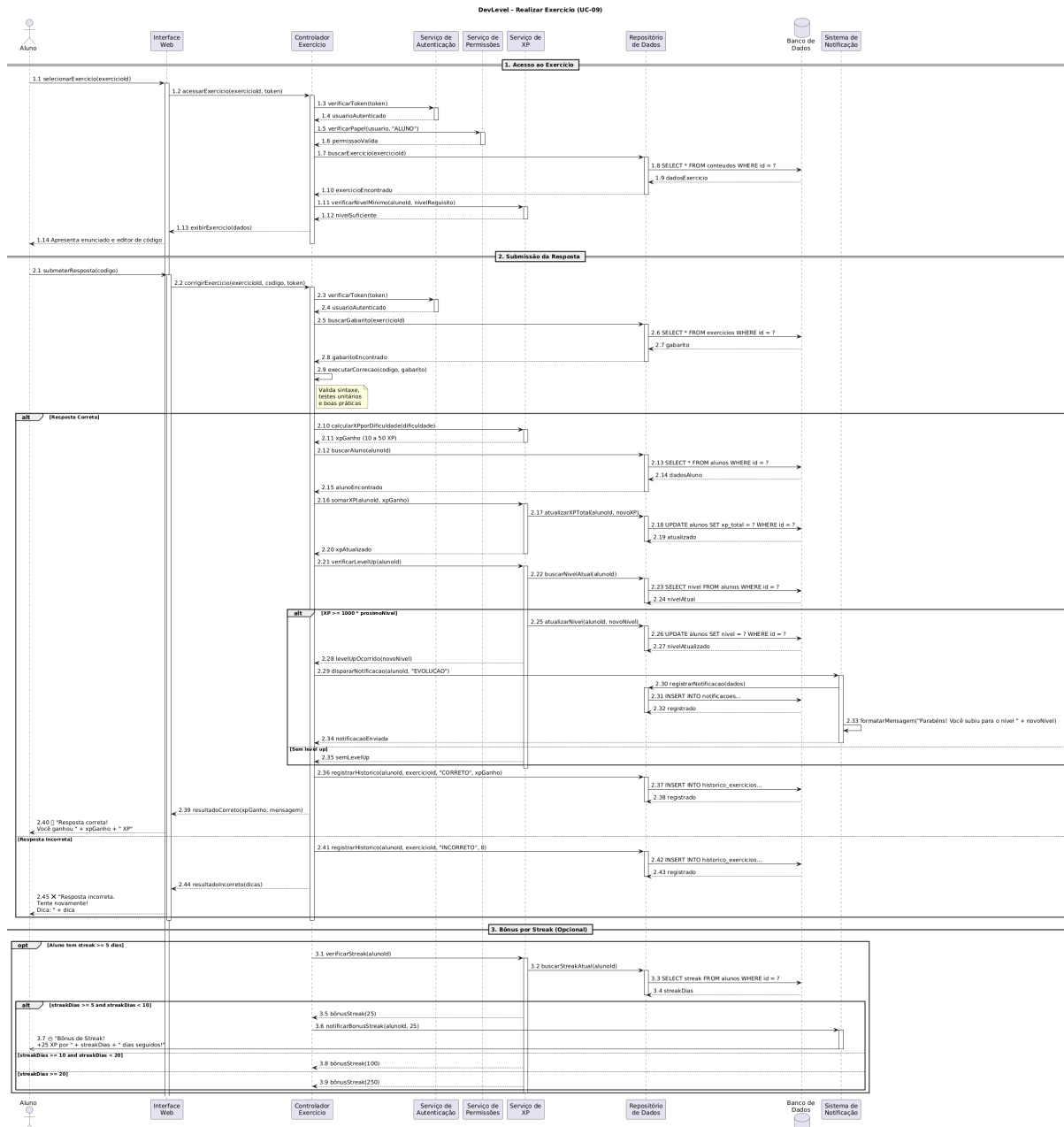


Figura - Diagrama de Sequência do Sistema - UC-09: Realizar Exercício

O diagrama a seguir detalha o processo de geração e exportação de relatórios pelo mentor, incluindo a seleção do tipo de relatório, configuração de parâmetros e formato, coleta e processamento de dados, geração do arquivo para download, e funcionalidade opcional de agendamento de relatórios periódicos.



Documentação de Projeto para o Sistema DevLevel Página

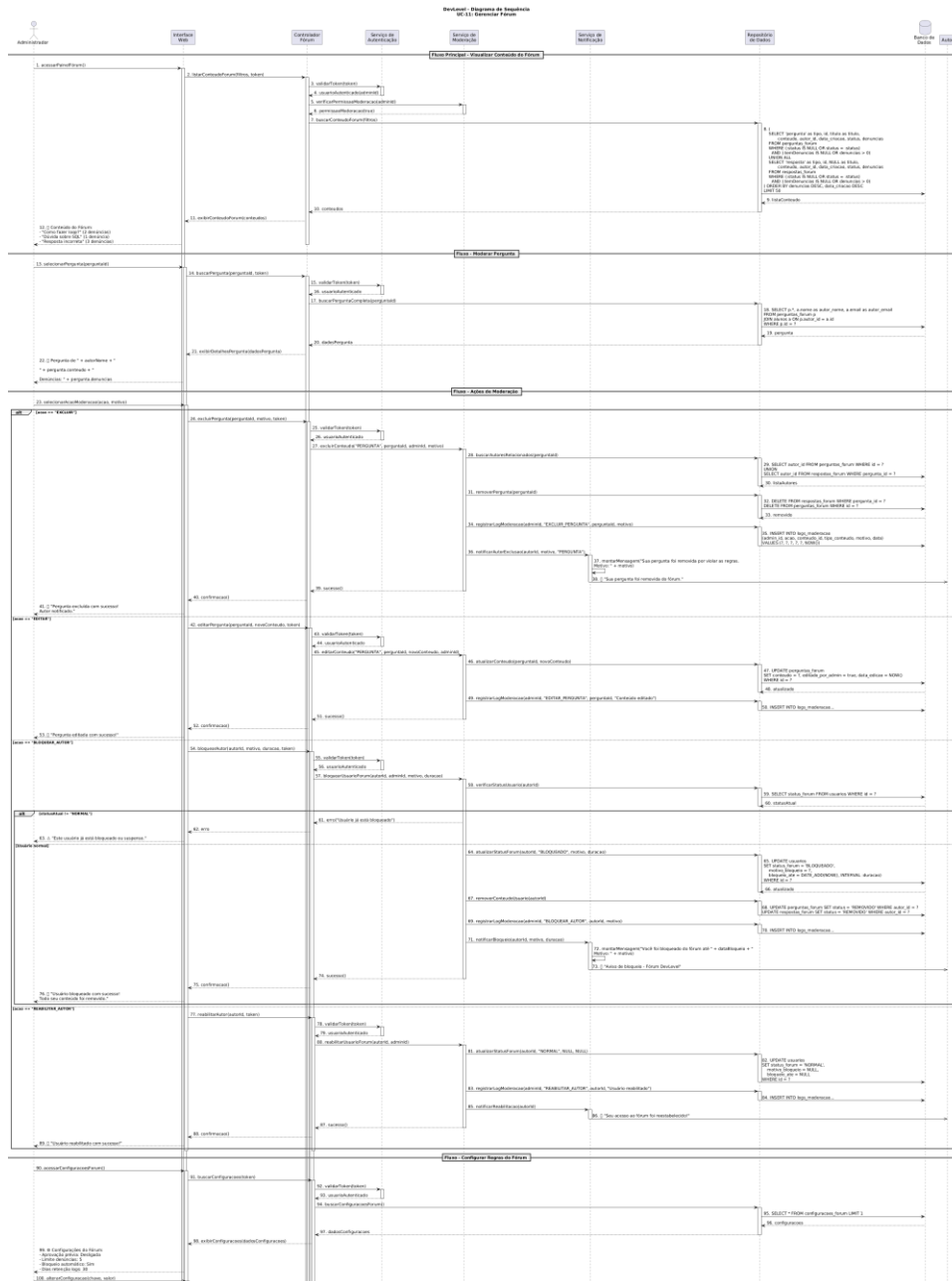


Figura - Diagrama de Sequência do Sistema - UC-11: Gerenciar Fórum

O diagrama a seguir detalha o processo de solicitação de mentoria pelo aluno, incluindo a busca por mentores disponíveis, verificação de disponibilidade e saldo de DevCoins, débito da moeda virtual, criação da solicitação, notificação do mentor, e acompanhamento do status pelo aluno.

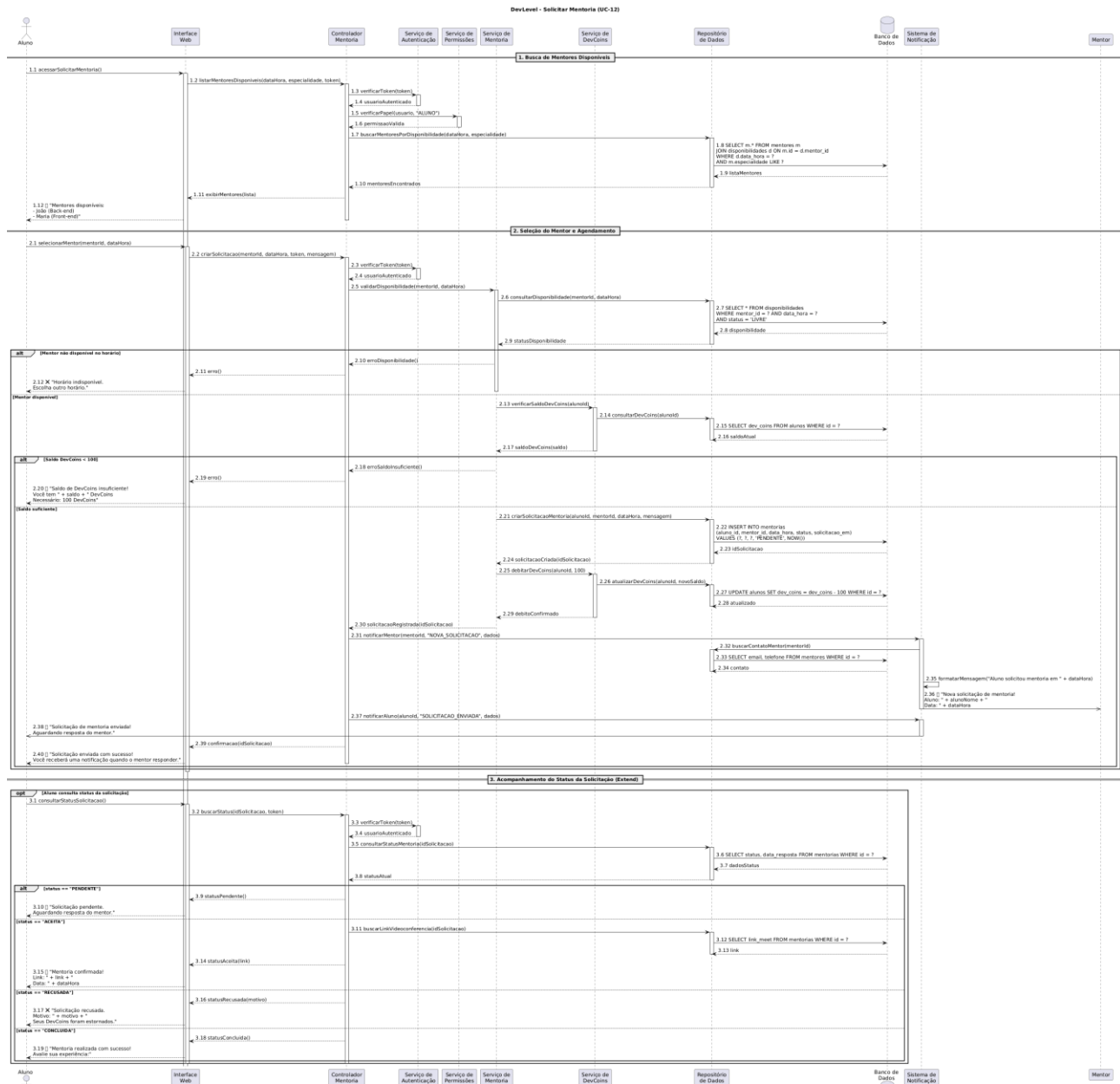


Figura 12 - Diagrama de Sequência do Sistema - UC-12: Solicitar Mentoria

O diagrama a seguir detalha as operações de gerenciamento de badges pelo administrador, incluindo criação com upload de ícone, edição, ativação, desativação e remoção de badges, bem como a visualização do catálogo completo com estatísticas de concessão.



O diagrama a seguir detalha o gerenciamento de solicitações de mentoria pelo mentor, incluindo visualização de solicitações pendentes, aceitação com geração de link de videoconferência, recusa com estorno de DevCoins e notificação do aluno, e remarcação ou cancelamento de sessões agendadas.

Documentação de Projeto para o Sistema DevLevel Página

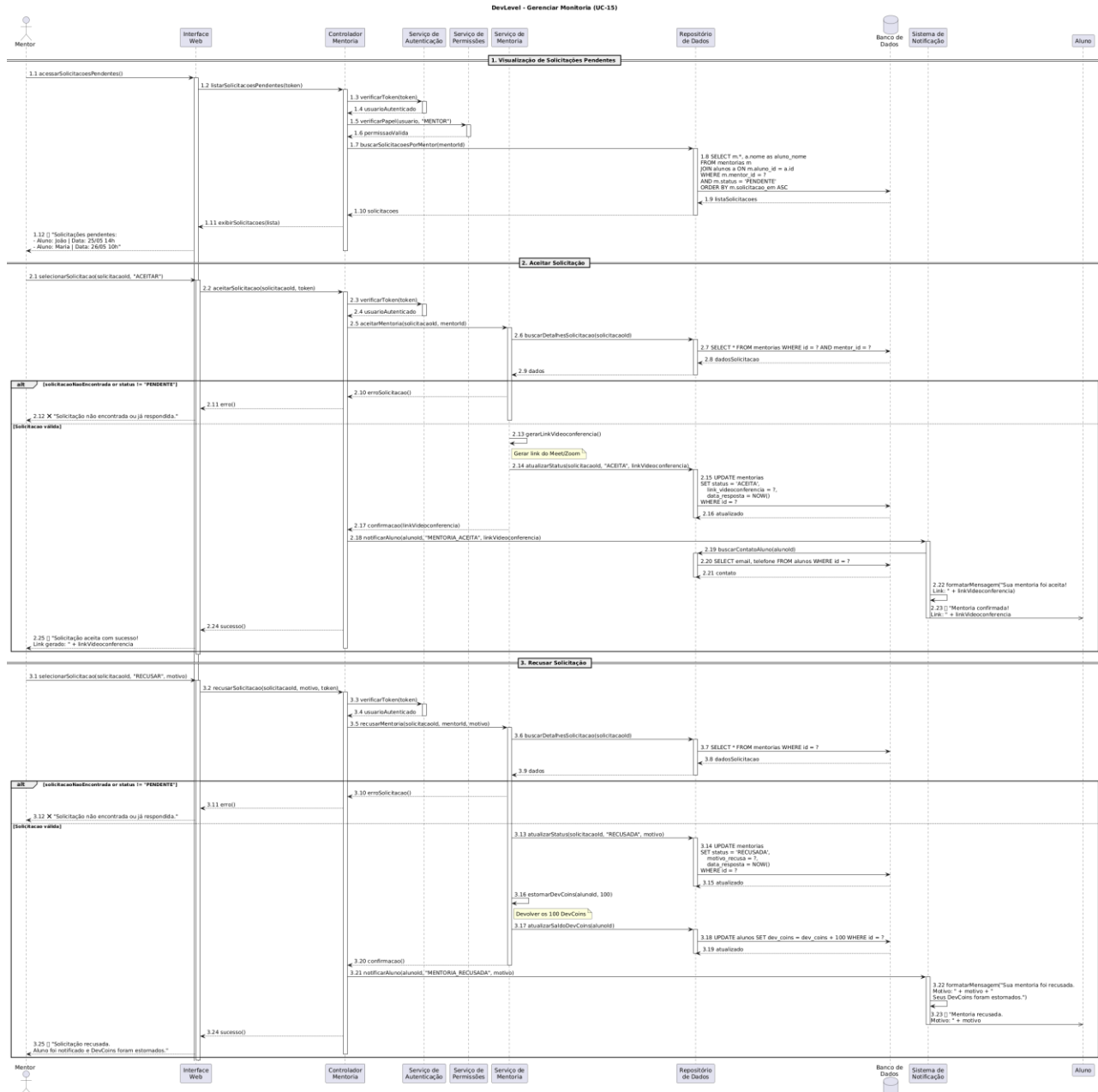


Figura - Diagrama de Sequência do Sistema - UC-15: Solicitar Mentoria

3.5 Diagramas de Comunicação

3.5.1 Diagrama de Comunicação - Solicitação de Mentoria

O diagrama de comunicação do caso de uso **UC-12 – Solicitar Mentoria** representa a interação entre o aluno e os componentes internos do sistema DevLevel durante o processo de solicitação de uma mentoria. O objetivo principal desse fluxo é permitir que o aluno envie uma solicitação de acompanhamento ou suporte acadêmico, garantindo autenticação, processamento da solicitação e persistência das informações no sistema.

O fluxo inicia quando o ator **Aluno** preenche e envia os dados da solicitação por meio da InterfaceWeb. Em seguida, a interface encaminha os dados ao ControladorMentoria, responsável por coordenar o fluxo do caso de uso.

Antes da criação da solicitação, o controlador realiza a validação do token de autenticação através do ServicoAutenticacao, garantindo que apenas usuários autenticados possam acessar a funcionalidade. Após a validação bem-sucedida, o ServicoMentoria executa a lógica de negócio relacionada à criação da solicitação.

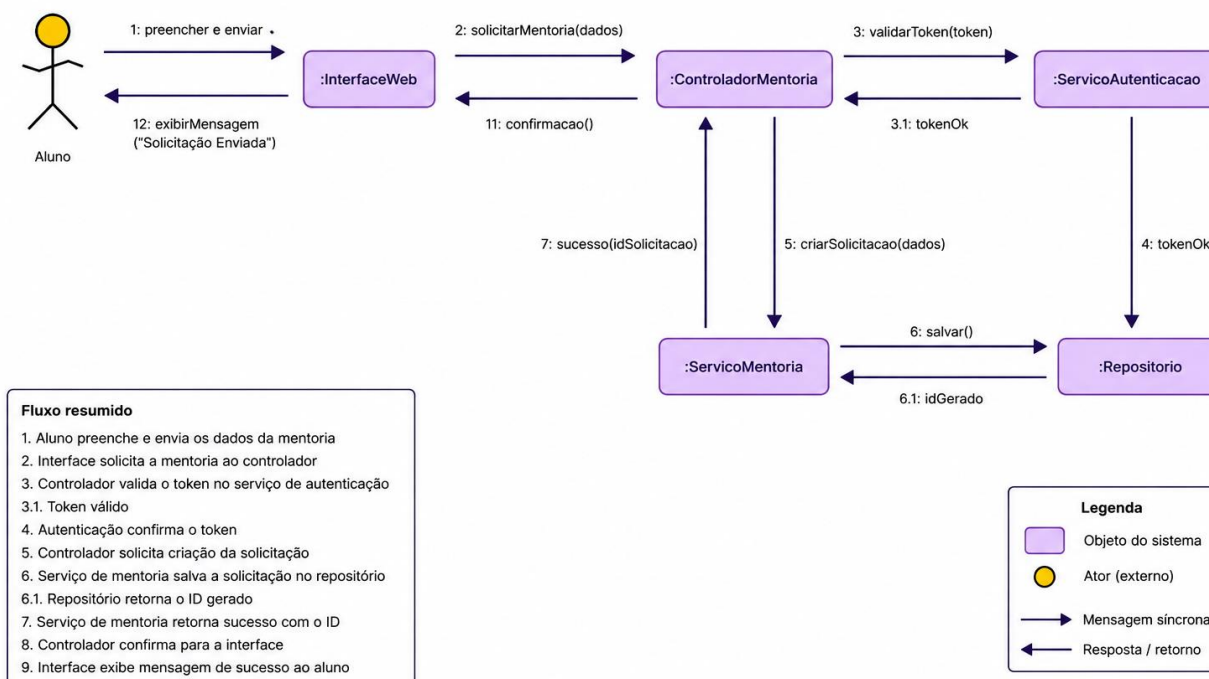
Posteriormente, o serviço realiza a persistência das informações no Repositorio, responsável pelo armazenamento dos dados no banco de dados. Após o salvamento, o repositório retorna o identificador da solicitação criada, permitindo que o sistema confirme o sucesso da operação.

Por fim, o controlador envia uma confirmação para a interface, que exibe ao aluno uma mensagem indicando que a solicitação de mentoria foi realizada com sucesso.

Este diagrama evidencia a separação de responsabilidades entre as camadas de interface, controle, serviços e persistência, seguindo princípios de arquitetura em camadas e promovendo maior organização, manutenção e escalabilidade do sistema DevLevel

Diagrama de Comunicação

UC-12 – Solicitar Mentoria



3.6.1 Diagrama de Comunicação - Realizar Exercício

O diagrama de comunicação do caso de uso **UC-09 – Realizar Exercício** representa a interação entre o aluno e os componentes internos do sistema DevLevel durante o processo de resolução de exercícios da plataforma. Esse fluxo é um dos principais do sistema, pois está diretamente relacionado às mecânicas de gamificação, evolução de nível e acompanhamento do desempenho do aluno.

O processo inicia quando o ator **Aluno** seleciona um exercício na InterfaceExercicio e envia suas respostas para o sistema. A interface encaminha as informações ao ControladorExercicio, responsável por coordenar o fluxo da operação.

Em seguida, o controlador solicita ao ServicoExercicio a correção das respostas enviadas. O serviço executa as regras de negócio relacionadas ao exercício, realizando o cálculo da pontuação obtida e verificando se o exercício foi concluído corretamente.

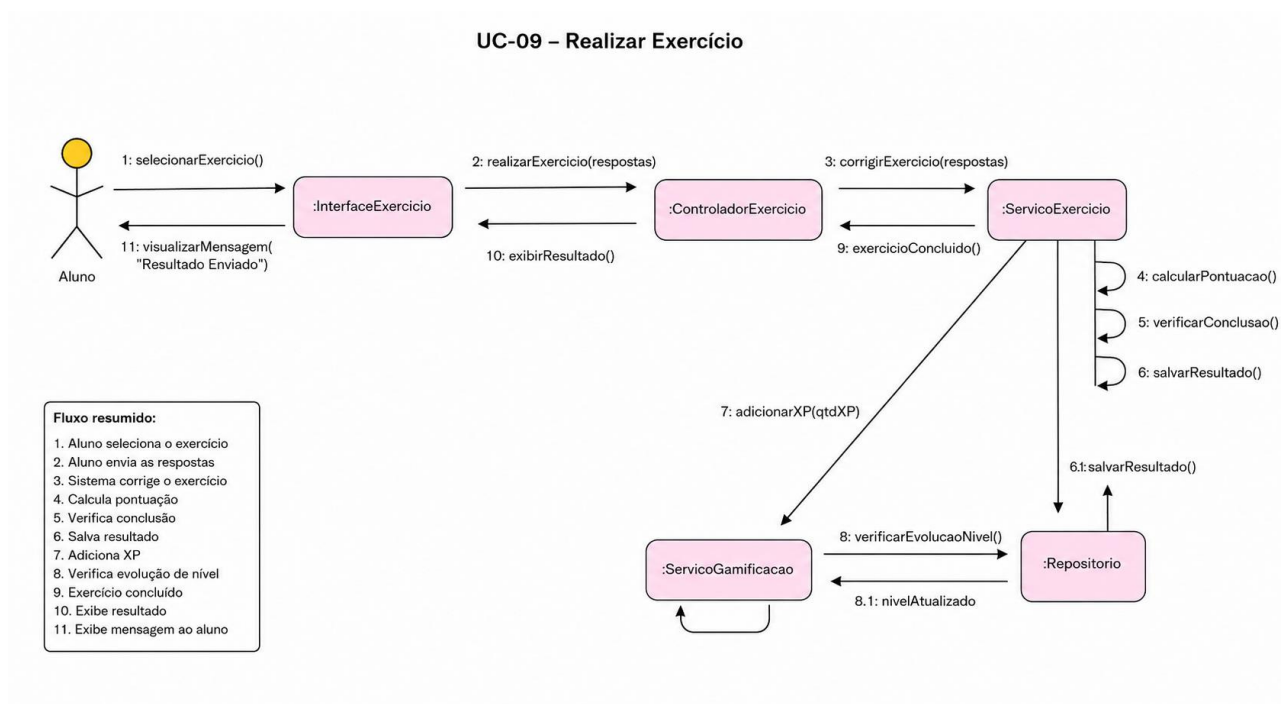
Após o processamento, o resultado é armazenado no Repositorio, responsável pela persistência das informações no banco de dados. O repositório retorna uma confirmação indicando que o resultado foi salvo com sucesso.

Posteriormente, o ServicoExercicio interage com o ServicoGamificacao, responsável pelas funcionalidades de gamificação do sistema. Nesse momento, são adicionados pontos de experiência

(XP) ao perfil do aluno e realizada a verificação de evolução de nível, permitindo desbloqueio progressivo de funcionalidades e conteúdos da plataforma.

Após a atualização das informações de progresso, o sistema retorna ao controlador a confirmação de conclusão do exercício. O controlador então solicita à interface a exibição do resultado, apresentando ao aluno uma mensagem informando que o exercício foi concluído e processado com sucesso.

Este diagrama demonstra a comunicação entre as camadas de interface, controle, serviços e persistência do DevLevel, além de evidenciar a integração das funcionalidades de gamificação ao fluxo principal do sistema. A estrutura adotada favorece organização, reutilização de componentes, separação de responsabilidades e escalabilidade da aplicação.



3.6 Diagramas de Estados

Nesta subseção são apresentados os diagramas de estados das principais entidades do sistema DevLevel, seguindo a notação UML 2.0. Cada diagrama representa o ciclo de vida de uma entidade, incluindo seus estados, transições, guardas condicionais (entre colchetes) e ações (precedidas por barra).

3.6.1 Diagrama de Estados - Solicitação de Mentoria

O diagrama a seguir representa o ciclo de vida de uma solicitação de mentoria no sistema, desde a criação pelo aluno até os estados finais de conclusão, recusa ou cancelamento. Inclui as transições de aceitação com geração de link, recusa com estorno de DevCoins, e cancelamento antes ou após a aceitação.

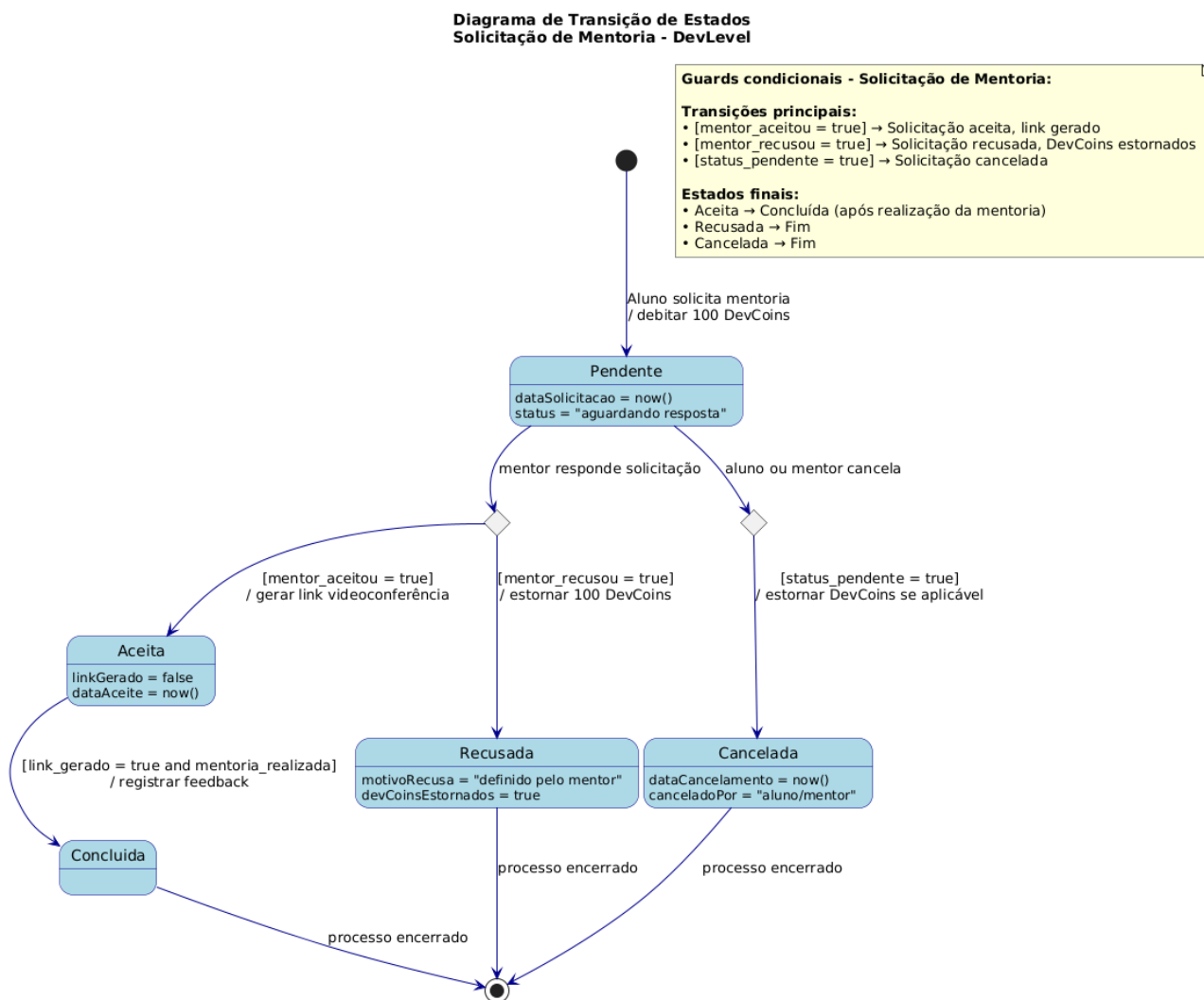


Figura 3.6.1: Diagrama de Estados da Solicitação de Mentoria

3.6.2 Diagrama de Estados - Missão Diária

O diagrama a seguir representa o ciclo de vida das missões diárias atribuídas aos alunos. Inclui a geração automática à meia-noite, os estados de disponibilidade, execução, conclusão com ganho de XP e DevCoins, expiração ao final do dia, e a funcionalidade de troca (reroll) mediante pagamento

de XP.

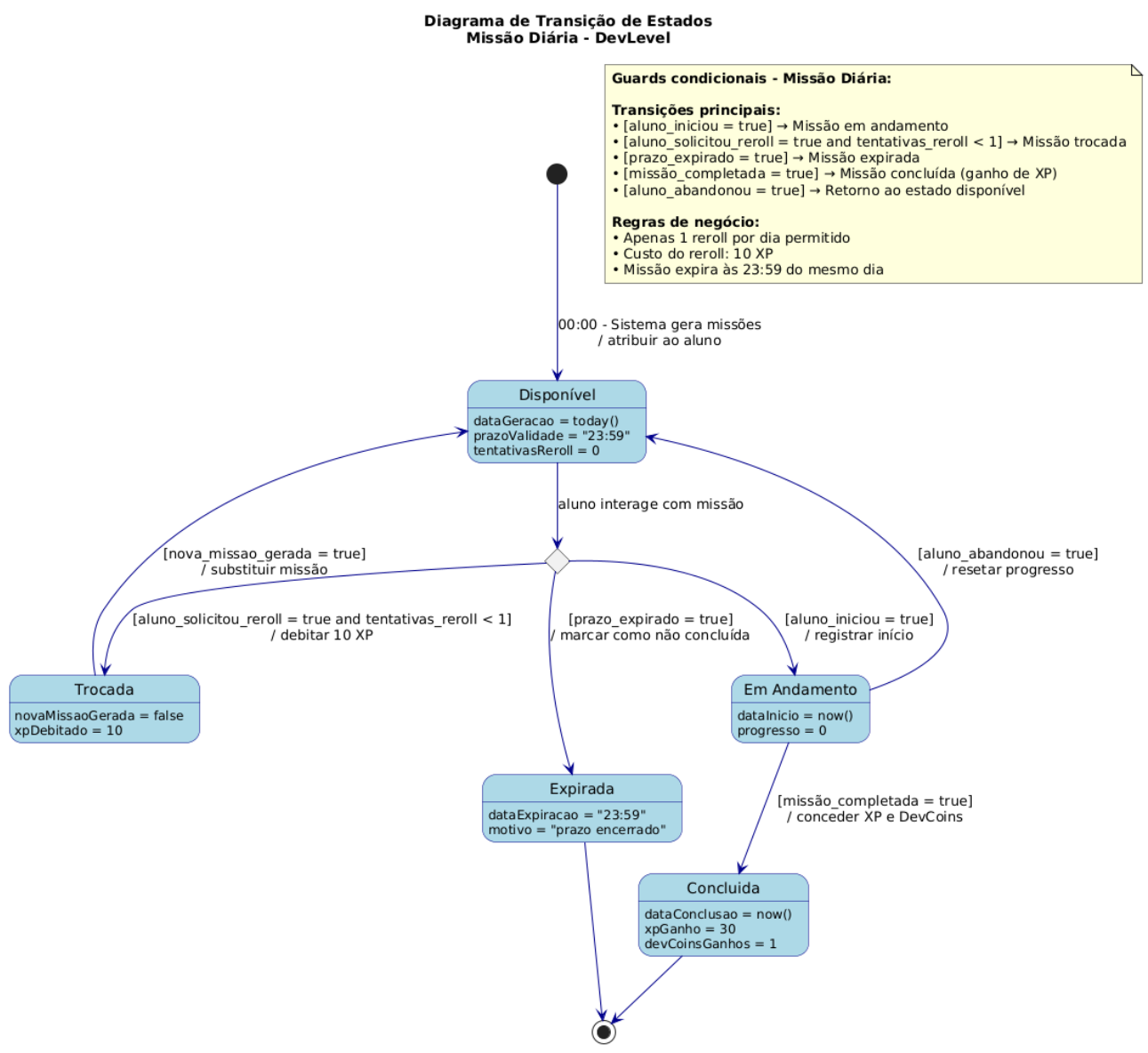
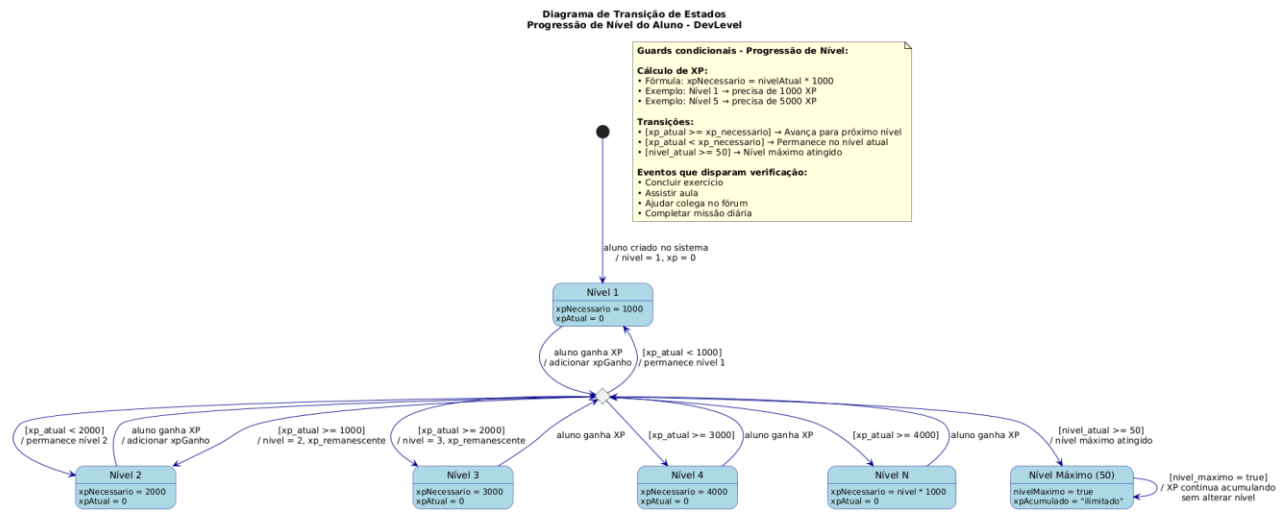


Figura 3.6.2: Diagrama de Estados da Missão Diária

3.6.3 Diagrama de Estados - Nível do Aluno

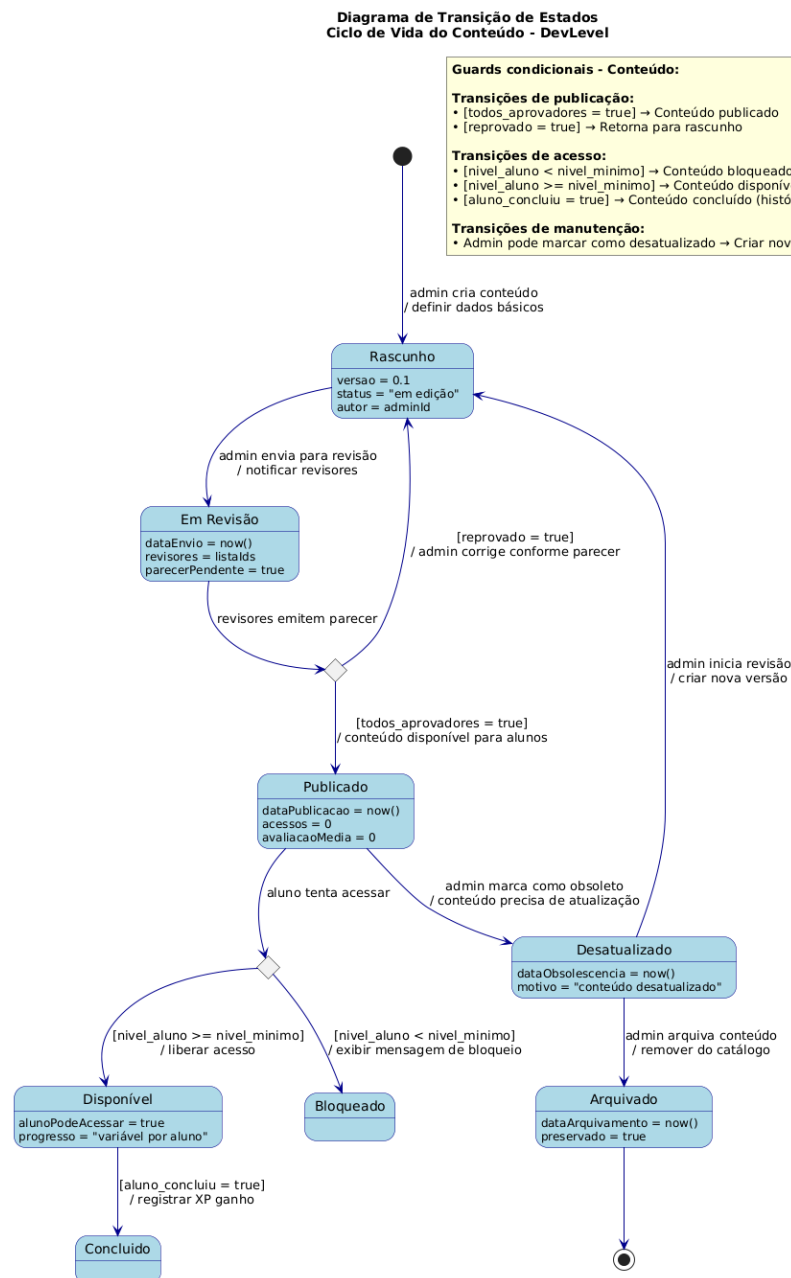
O diagrama a seguir representa a progressão de níveis do aluno no sistema de gamificação. A cada 1000 XP acumulados, o aluno avança para o próximo nível, até o nível máximo 50. O diagrama mostra a verificação contínua de XP e as transições entre níveis.



****Figura 3.6.3: Diagrama de Estados do Nível do Aluno****

3.6.4 Diagrama de Estados - Conteúdo

O diagrama a seguir representa o ciclo de vida dos conteúdos didáticos (aulas e exercícios) no sistema. Inclui as etapas de criação em rascunho, envio para revisão, aprovação e publicação, além da possibilidade de desatualização, arquivamento e os estados de acesso pelo aluno (bloqueado, disponível, concluído).



****Figura 3.6.4: Diagrama de Estados do Conteúdo****

3.6.5 Diagrama de Estados – Badge

O diagrama a seguir representa o ciclo de vida das badges (conquistas) no sistema. Inclui a criação pelo administrador, ativação para concessão aos alunos, concessão automática quando o critério é atendido, desativação, reativação e remoção definitiva.

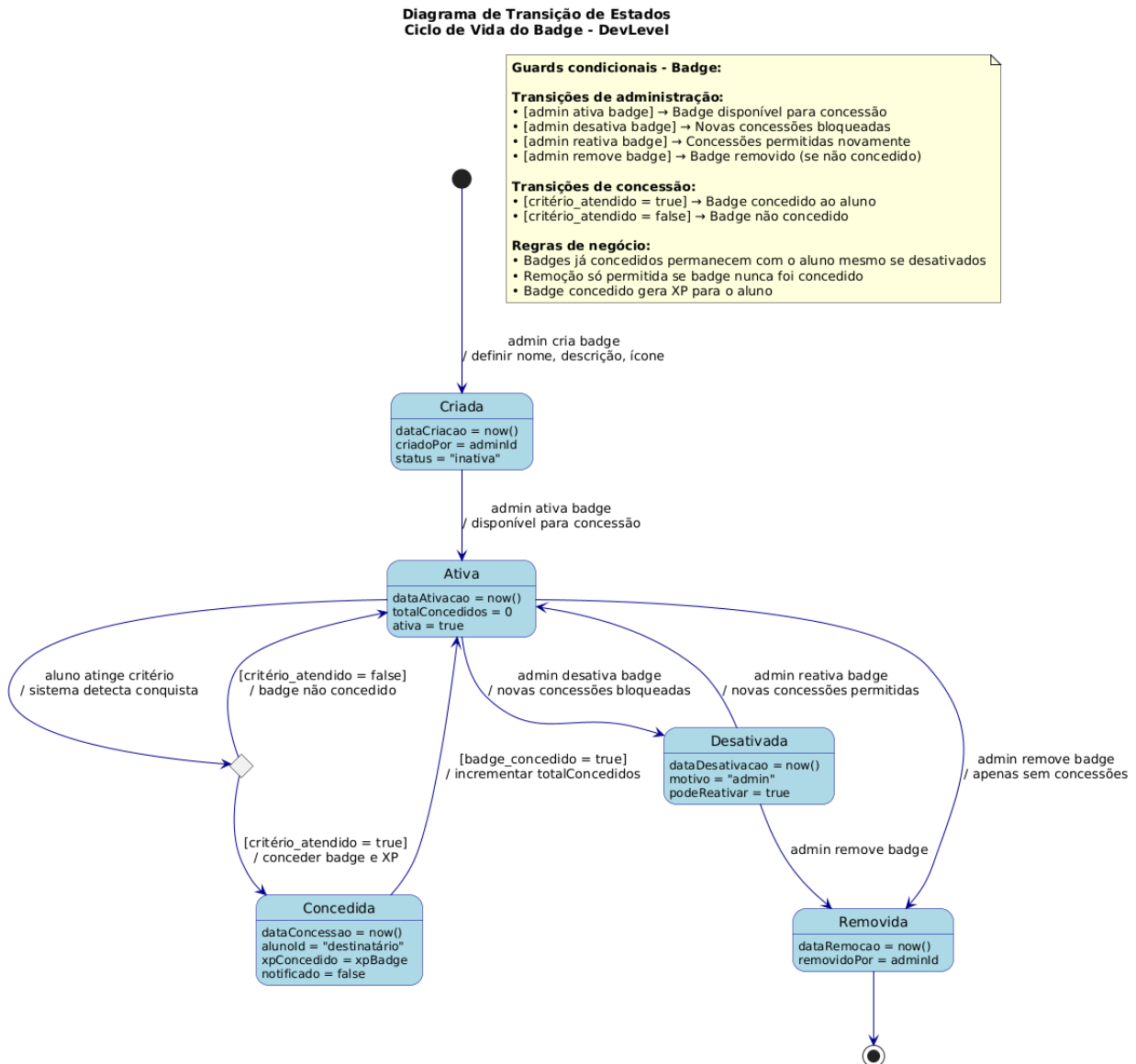


Figura 3.6.5: Diagrama de Estados do Badge

4. Modelos de Dados

Nesta seção são apresentados os esquemas de banco de dados do sistema DevLevel, baseados no Diagrama Entidade-Relacionamento (DER) derivado do diagrama de classes. O modelo de dados representa a estrutura lógica de persistência das informações do sistema.

4.1 Tecnologia de Persistência

O sistema utiliza PostgreSQL 16 como sistema gerenciador de banco de dados relacional, hospedado em ambiente de nuvem. As estratégias de mapeamento adotadas incluem o tratamento da herança por meio de tabelas separadas para cada tipo de usuário (Aluno, Mentor e Administrador), compartilhando a chave primária da tabela base Usuario. Os valores de domínio fixos, como tipos de notificação e status de mentoria, são armazenados como textos para facilitar a leitura e a geração de relatórios. Relacionamentos muitos-para-muitos, como a relação entre Aluno e Badge, são implementados por meio de tabelas associativas. Índices são criados em campos frequentemente utilizados em consultas, como email, status e datas, para otimizar o desempenho das operações de busca.

4.2 Diagrama Entidade-Relacionamento

O diagrama a seguir apresenta o modelo entidade-relacionamento do banco de dados do sistema DevLevel, mostrando todas as entidades, seus atributos e os relacionamentos com suas respectivas cardinalidades.

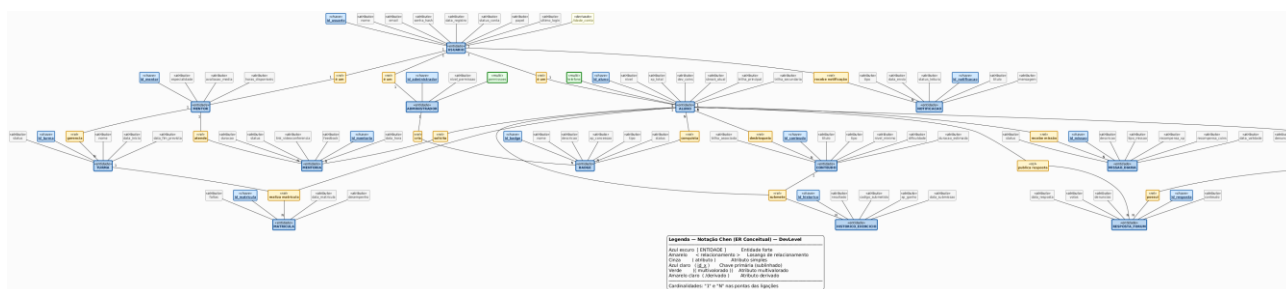


Figura 4.1: Diagrama Entidade-Relacionamento (DER) do Sistema DevLevel

4.3 Esquema do Banco de Dados

O banco de dados do sistema DevLevel é composto por dezoito tabelas, organizadas nas seguintes categorias: usuários e perfis, turmas e matrículas, gamificação, conteúdos, missões diárias, mentorias, fórum, notificações e logs.

4.3.1 Tabelas de Usuários e Perfis

A tabela usuarios é a tabela base para todos os usuários do sistema. Ela armazena o identificador único como chave primária, nome completo, endereço de e-mail com restrição de unicidade, hash da senha, data de registro, status da conta (ativo, inativo ou bloqueado), papel do usuário (aluno, mentor ou administrador) e data do último login.

A partir desta tabela base, são derivadas três tabelas específicas. A tabela alunos estende a tabela usuarios e armazena informações específicas dos estudantes, incluindo nível atual (que varia de 1 a 50), experiência total acumulada (XP), saldo de DevCoins, sequência atual de dias com atividade (streak), além das trilhas de especialização principal e secundária.

A tabela mentores armazena dados dos instrutores, como especialidade profissional, carga horária semanal disponível para mentorias e avaliação média recebida dos alunos.

A tabela administradores contém informações dos gestores do sistema, incluindo o nível de permissão (super administrador, administrador de conteúdo ou administrador de suporte) e as permissões especiais concedidas.

4.3.2 Tabelas de Turmas e Matrículas

A tabela turmas representa os agrupamentos de alunos sob responsabilidade de um mentor. Ela armazena o identificador único da turma, nome, referência ao mentor responsável, data de início, data prevista de término e status (ativa, concluída ou cancelada).

A tabela matriculas funciona como uma classe associativa que estabelece o relacionamento entre alunos e turmas. Ela contém referências ao aluno e à turma, a data da matrícula, o desempenho do aluno na turma (representado como um valor percentual) e o número de faltas. Para garantir a integridade referencial, existe uma restrição de unicidade composta para evitar matrículas duplicadas.

4.3.3 Tabelas de Gamificação

A tabela badges armazena as conquistas disponíveis no sistema, contendo identificador único, nome com restrição de unicidade, descrição detalhada, URL do ícone, critério de desbloqueio, quantidade de XP concedida ao ser desbloqueada, tipo da badge (nível, conquista ou especial), status (ativa, desativada ou removida) e referência ao administrador que a criou.

Para registrar quais badges cada aluno conquistou, existe a tabela associativa alunos_badges, que armazena o par de identificadores do aluno e do badge como chave primária composta, a data da conquista e, opcionalmente, a referência ao mentor que validou a conquista em casos de validação manual.

4.3.4 Tabelas de Conteúdos

A tabela conteúdos é a base para todo o material didático disponível na plataforma. Ela contém identificador único, título, descrição, tipo (aula, exercício, artigo ou vídeo), URL do recurso, nível mínimo necessário para desbloqueio, trilha de especialização associada, duração estimada em minutos, dificuldade (básico, intermediário ou avançado) e status (rascunho, publicado, desatualizado ou arquivado).

A tabela conteúdos_desbloqueados registra quais conteúdos foram desbloqueados por cada aluno, armazenando as referências ao aluno e ao conteúdo, e a data do desbloqueio.

A tabela historico_exercicios armazena o histórico completo de submissões de exercícios, incluindo referências ao aluno e ao exercício, o código enviado, o resultado da correção (correto ou incorreto), o XP ganho e a data da submissão.

4.3.5 Tabela de Missões Diárias

A tabela missoes_diarias representa os desafios diários atribuídos a cada aluno. Ela contém identificador único, referência ao aluno destinatário, descrição da missão, tipo de missão (exercício, aula, ajuda ou revisão), recompensa em XP, recompensa em DevCoins, data de validade para conclusão e status (pendente, concluída, expirada ou trocada).

4.3.6 Tabelas de Mentorias

A tabela mentorias gerencia todo o ciclo de vida das solicitações de mentoria. Ela armazena o identificador único da solicitação, referências ao aluno solicitante e ao mentor designado, data e hora agendada, duração em minutos, status (pendente, aceita, recusada, concluída ou cancelada), link para videoconferência gerado automaticamente, motivo da recusa, feedback do aluno, data da solicitação e data da resposta do mentor.

4.3.7 Tabelas de Fórum

A tabela perguntas_forum armazena as perguntas feitas pelos alunos. Ela contém identificador único, referência ao aluno autor, título, conteúdo da pergunta, data de criação, status (pendente, respondida, concluída ou removida), número de denúncias recebidas e referência opcional à melhor resposta.

A tabela `respostas_forum` armazena as respostas fornecidas pelos alunos às perguntas. Ela contém identificador único, referência à pergunta respondida, referência ao aluno autor, conteúdo da resposta, data da resposta, número de votos úteis recebidos e número de denúncias.

4.3.8 Tabelas de Notificações e Logs

A tabela `notificacoes` registra todas as comunicações enviadas aos usuários. Ela armazena identificador único, referência ao usuário destinatário, título, mensagem, tipo de notificação (evolução, mentoria, conquista ou alerta), data de envio, status de leitura e data da leitura.

4.4 Índices e Otimização

Para garantir o desempenho adequado do sistema, diversos índices são criados nas tabelas mais frequentemente acessadas. Um índice único no campo `email` da tabela `usuarios` acelera as consultas de login. Um índice no campo `nivel` da tabela `alunos` otimiza a ordenação em rankings e filtros por nível. Índices nos campos `aluno_id` e `turma_id` da tabela `matriculas` aceleram as consultas de turmas de um aluno e de alunos de uma turma. Um índice composto nos campos `mentor_id` e `status` da tabela `mentorias` otimiza a listagem de solicitações pendentes para um mentor específico. Um índice no campo `data_submissao` da tabela `historico_exercicios` facilita a geração de relatórios temporais. Um índice composto nos campos `usuario_id` e `status_leitura` da tabela `notificacoes` permite consultas eficientes de notificações não lidas. Um índice no campo `data_validade` da tabela `missoes_diarias` acelera a rotina de limpeza de missões expiradas.

4.5 Resumo das Tabelas

O modelo de dados do sistema DevLevel é composto pelas seguintes tabelas: `usuarios`, `alunos`, `mentores`, `administradores`, `turmas`, `matriculas`, `badges`, `alunos_badges`, `conteudos`, `conteudos_desbloqueados`, `historico_exercicios`, `missoes_diarias`, `mentorias`, `perguntas_forum`, `respostas_forum`, `notificacoes`. Totalizam dezesseis tabelas que suportam todas as funcionalidades do sistema, garantindo integridade referencial, consistência dos dados e desempenho adequado por meio de índices estrategicamente posicionados nas colunas mais consultadas.